源码驱动的框架解剖

Claude Code 游戏工作室

从提示词到可验收流水线

参考本地 SAM 系列的讲解方法
前言 · 17 章 · 附录

CCGS 1.2.0
05ad7c8a5b8aa2aa86bbd3439ae7cf7295cac418
采集日期 2026-10-31

独立分析，非官方文档。静态源码研究、隔离脚本实验与离线教学验证，不代表生产压测或完整游戏开发。

全书目录

1. 前言：先看一条真实的工作流
2. 需求：单人开发为什么需要组织
3. 总体架构：一个会话，四种约束
4. 入门：`/start` 如何把空白仓库变成项目
5. 配置求值：一个 rigor 如何长出六个设置
6. 角色系统：49 个 Agent 如何分工
7. Skill 协议：一个命令从解析到交接
8. 产物模型：故事、需求、ADR 三张网
9. 实施链路：`/dev-story` 怎么把故事变成代码
10. 验收：`/story-done` 如何区分事实与断言
11. Hook：十五个检查点里谁真的能拦住
12. 注册表：跨文档事实如何不漂移
13. 协作：团队流水线与它的塌缩
14. 引擎适配：代码根不是一个常量
15. 上下文与恢复：把文件当作记忆
16. 安全边界：能聊天不等于能管理
17. 验证与演进：这门框架怎么证明自己没坏
18. 最小实验：用一百多行 Python 复刻四条约束
19. 附录：源码索引、术语与核验路线

00

前言：先看一条真实的工作流

一个周末，你打开终端，克隆下这个仓库，输入 `/start` 。它会问你有没有游戏想法、打算用什么引擎，然后建议一个流程强度。你说"有个潜行小游戏的念头，还没写任何东西"。它记下 `modes.rigor: minimal` ，告诉你四步之后就能写出第一行游戏代码。

接着你输入 `/brainstorm` ，写下几段设计； `/create-stories` 把这份简短设计拆成可执行的故事； `/dev-story` 把其中一个故事交给对应的程序员角色，要求它先读验收标准，再写代码和测试。故事写完， `/story-done` 逐条核对验收标准，把没有证据的条目标成 UNTESTED，而不是"基本完成"。

这条路径是这个仓库想要交付的东西。它的价值不在于"49 个 Agent"这个数字，而在于每一步都规定了：读什么、写什么、什么时候可以停、停不下来时怎么报告。

这本书讲什么

Claude Code Game Studios（下称 CCGS）不是游戏引擎，也不是独立运行的服务。它是一套放在 Git 仓库里的工作流资产：Markdown 写的角色、带前置元数据的技能命令、Bash 写的检查脚本、YAML 写的项目状态。宿主是 Claude Code，运行时是模型的会话。仓库自己不产生一个进程。

因此读懂它的方式和读一个 Web 服务不同。没有请求入口可以顺着追，没有数据库 schema 可以反推。你需要看清四条线索：

1. 一段文字如何进入模型上下文。入口是 `CLAUDE.md` 、技能正文、Agent 提示词、按需读取的参考文档。
2. 一个设置如何解析成实际生效的值。入口是 `project.yaml` 、`project.local.yaml` 、旧版镜像文件、rigor 展开、内置默认。
3. 一个动作在什么条件下被允许或阻断。入口是 `.claude/settings.json` 的权限和 Hook 注册表，以及 `validate-*.sh` 的退出码。
4. 一次交付如何被判定为完成。入口是故事模板、 `/dev-story` 、 `/story-done` 和证据文件。

四者叠起来才构成"工作室"。只抄提示词，拿到的是一个会写代码的聊天助手；把配置解析、Hook、验收一起搬走，才拿到一个能反复运转的流程。

研究边界

本书基于公开仓库的一次固定快照：

- 仓库： `Donchitos/Claude-Code-Game-Studios`
- 提交： `05ad7c8a5b8aa2aa86bbd3439ae7cf7295cac418` （2026-10-19, Release 1.2.0）
- 许可：MIT
- 采集日期：2026-10-31

书中出现的统计都来自这次快照的实测：`.claude/agents/` 下有 49 个角色文件，`.claude/skills/` 下有 74 个 `SKILL.md`，`.claude/rules/` 下有 14 个规则文件，`.claude/hooks/` 下有 18 个脚本，其中 15 个被 `settings.json` 注册为 Hook 命令，`docs/templates/` 下有 71 个模板文件。

写这本书时做了三类工作：静态阅读源码；在隔离目录里直接运行上游脚本，观察真实退出码和输出；用一份独立编写的 Python 模型复刻它的核心约束。没有做的是：完整跑通一场真实的游戏开发、统计模型成本、评估产出的游戏质量。这三件事需要真实项目和真实计费，静态研究无法替代。

凡是涉及"这个机制在生产里表现如何"的判断，书中都会标明是推断；凡是脚本实际跑出来的结果，会给出命令和输出。

一个必须先说的结论

CCGS 的很多规则写在提示词里，而不是写在代码里。提示词是建议，Hook 才是强制。区分这两者，决定你对这套框架的信任程度。

举个具体例子。`validate-write.sh` 在每次写入前检查被保护的文件，比如故事文件是否被覆盖、配置是否写错位置、原型目录是否被生产代码引用。它发现问题时打印警告，然后 `exit 0`。也就是说，它提醒模型，但不阻止写入。同一份 `settings.json` 里，`validate-read.sh` 在拒绝整篇读取大型设计文档时用 `exit 2`，这一次是硬阻断。

把这两种情况混为一谈，就会得出"有 Hook 所以安全"的错觉。书里第 10 章会把每个 Hook 的退出行为和实际保护范围拆开。

阅读路线

第一次接触，按顺序读第 1 到第 3 章，先建立"这是什么、怎么启动"的整体印象；然后跳到第 8、9 章，看一次实施和验收的完整链路，这是全仓库最值得借鉴的部分；回头读第 4、5、6、7 章补齐配置和角色系统；第 10 到 17 章按兴趣选读，涉及 Hook、注册表、引擎适配、恢复、安全和演进；第 18 章是动手环节，用一百多行 Python 把故事状态机、配置解析、验收判定和证据过期检查跑一遍。

每章固定结构：一个具体场景开头，源码入口，机制拆解，常见误解，三道练习与答案提示。练习不需要 Claude Code，也不需要游戏引擎，能在本机终端完成。

读完之后，你应该能回答三个问题：这套框架在哪一层真正约束了模型？它的哪些规则只是自觉？如果要给自己的项目搭类似流程，哪几个部件值得抄，哪几个可以删。

01

需求：单人开发为什么需要组织

场景

周五晚上，你有四个小时。想做一款潜行小游戏：玩家躲守卫、撬门、带走证据。脑子里有画面，手上没代码。打开 Claude Code，打字"帮我做个潜行游戏"，它可能会给你一个能跑的原型：一个方块、一个巡逻的圆点、一个计时器。三小时后你发现门禁逻辑写在角色控制脚本里，守卫的视野角度是硬编码的 `90`，测试没有，设计文档没有，第二天你已经不记得为什么把发现阈值设成 `0.7`。

这不是模型能力问题。是你把"设计、实现、验收、记账"四件事压进了一次对话，而对话没有留下可检查的中间状态。

CCGS 的起点就是这个失败模式。它的 README 写得直白：一次会话很有力，但没有结构，没人阻止你硬编码魔法数字、跳过设计文档、写面条代码，也没有 QA 和设计评审。

需求从哪里来

仓库把需求分散在三处，读的时候要合起来看。

第一处是根目录 `CLAUDE.md`。它是每个会话都会加载的常驻说明，声明了技术栈占位符、目录结构、协作协议，以及两条被特意点名"承重"的约定：`production/session-state/active.md` 是会话检查点，`.claude/scripts/` 里的辅助脚本只输出观察，不输出判决。

第二处是 `docs/COLLABORATIVE-DESIGN-PRINCIPLE.md`。这份文档给出整仓最核心的交互模型：

```
Agent = 专家顾问
User  = 创意总监，最终决策者
```

并规定每次交互走五步：提问 → 给选项 → 用户决策 → 出草稿 → 用户批准。文档还专门列了错误模型：Agent 自己设计、自己写文件、用户只说一句"做个游戏"然后等着。它要防的就是这个。

第三处是 `.claude/docs/coding-standards.md` 和 `.claude/rules/` 下 14 个按路径生效的规则文件。它们管的是"代码写成什么样"：游戏数值必须来自数据文件，状态机必须有显式转移表，UI 不得直接持有游戏状态。

三处合起来，需求可以归纳成四条：

1. 决策权留在人手里。模型可以提议，不能替你拍板。
2. 中间产物落盘。设计、决策、进度都要有文件，不能只存在于对话里。
3. 实现前先有可验收的标准。故事文件里的验收条件就是这条的载体。
4. 完成要有证据。测试文件、截图、运行结果，缺了就要标出来。

一个需求如何变成一条规则

拿"不要硬编码魔法数字"举例。这句话在很多项目规范里都出现过，通常没人执行。CCGS 把它变成了三个层次：

第一层，写在角色提示词里。`.claude/agents/gameplay-programmer.md` 的职责第 2 条写着：所有玩法数值必须来自外部配置文件，设计师不碰代码也能调。代码标准里再写一遍：所有数值来自配置文件并带合理默认值。

第二层，写成按路径生效的规则。`.claude/rules/gameplay-code.md` 只在编辑玩法代码时加载。

第三层，交给脚本检查。`validate-commit.sh` 在暂存区里筛出代码根下的源文件，用一条正则找形如 `damage: 12`、`speed = 5` 的赋值，命中就打印 `CODE: <file> may contain hardcoded gameplay values. Use data files.`

第三层有一个值得注意的细节：这条告警不阻断提交。脚本里写明，只有暂存了 `assets/data/` 下的 JSON 且校验失败时才 `exit 2`。所以"不要硬编码"是一条有检测、无强制的规则。

这不是缺陷，而是分层设计的必然结果。数值是否该在数据文件里，有时需要人判断，比如一个只在一处使用的相机插值系数。把这种判断做成硬阻断，误报会迅速让人关掉检查。仓库选择提醒，把裁决留给评审环节。

估算一下成本

README 里有一张不寻常的表：四个游戏从同一份简报出发，在不同 rigor 下统计"写第一行游戏代码之前产出的文档数"。

rigor	首行代码前的文档数	耗时
minimal	1（一页简报）	0 分钟
standard	30	58 分钟

四个版本都产出了可运行、有测试的游戏。随后两位盲评者和一次真人试玩把 standard 版本排在最后。作者自己的结论是：多出的 29 份文档买到的是可追溯性，不是更好的游戏。

这个结论对读者很重要。它承认了流程的边际收益会下降，也因此把 `minimal` 设为默认值。任何讲这套框架的文章如果只强调"完整流程更专业"，都偏离了原作者的态度。

常见误解

误解一：这是给 3A 团队用的重型流程。实际默认值是最轻的一档，四步就能到代码，重型流程需要显式开启。

误解二：49 个 Agent 并行工作。默认 `team.size: individual`，团队流水线会把大多数角色折叠成一个执行者，第 13 章会展开。

误解三：文档越多质量越高。作者自己做了对照实验，结论相反。

练习

1. 打开 `CLAUDE.md`，找出其中被标为"承重"的两条约定，说明它们各自防止什么失败。
2. 读 `.claude/agents/gameplay-programmer.md` 的"Key Responsibilities"和"Code Standards"，列出至少三条既能写成提示词、又能写成脚本检查的规则。
3. 假设你要给一个已有项目加"禁止在 UI 层读游戏状态"的约束，请设计三层落地方式，并指出哪一层会真正阻断。

答案提示

1. 会话检查点防止上下文压缩或崩溃后丢失工作状态；辅助脚本只报告观察防止脚本用自己看不到的规则下判决，比如脚本不知道当前 rigor，却输出"通过"。
2. 例如：数值必须来自配置文件（可用正则扫描赋值）、状态机必须有显式转移表（可检查是否存在转移表结构）、UI 不得直接引用游戏状态（可 grep 导入路径）。
3. 提示词写进角色文件和 `CLAUDE.md`；路径规则写进 `.claude/rules/ui-code.md`；脚本检查放进 `validate-commit.sh`，但要先决定是警告还是阻断。如果要求真正阻断，必须让脚本 `exit 2`，并接受误报代价。

02

总体架构：一个会话，四种约束

场景

第一次打开这个仓库，目录树看起来像一份规范文档的合集：`.claude/` 下面有 agents、skills、hooks、rules、docs、scripts 六个子目录，根目录有 `CLAUDE.md` 和 `project.yaml`，再往下是 `design/`、`docs/`、`production/`、`studio-tests/`。没有 `package.json`，没有 `Dockerfile`，没有服务入口。

有人会问：这东西怎么运行？

答案是：它不自己运行。Claude Code 启动时读取当前目录的 `CLAUDE.md` 和设置文件，把一部分文本放进上下文，把一部分命令注册成事件回调。你输入斜杠命令时，Claude Code 找到对应的技能文件，把正文交给模型，模型再按正文里的步骤调用工具。所谓"工作室"，是这一串加载与调用的结果。

四种约束

把仓库里的所有机制按"约束力从软到硬"排队，只有四层。

第一层是常驻上下文。`CLAUDE.md` 用 `@path` 语法导入其他文档，例如 `@.claude/docs/directory-structure.md` 和 `@.claude/docs/coordination-rules.md`。每个会话开始都会带上这些内容，模型看得见，但可以忽略。这一层最软。

第二层是按需加载。技能的 `SKILL.md` 正文在调用时进入上下文；参考文档要模型主动读；Agent 提示词在派生子代理时加载。范围比第一层窄，内容更具体，仍然只是文字。

第三层是事件回调。`settings.json` 把 15 个命令挂到 8 类事件上：SessionStart、PreToolUse、PostToolUse、Notification、PreCompact、Stop、SubagentStart、SubagentStop。这些脚本能读到工具调用的完整参数，能打印警告，也能用特定退出码改变行为。

第四层是宿主权限。`settings.json` 的 `permissions` 段列出允许和拒绝的命令模式。例如 `Bash(rm -rf *)`、`Bash(git push --force*)`、`Read(**/.env*)` 在 deny 列表里。这一层由 Claude Code 自己执行，不依赖模型配合。

四层的强制力差别很大。第四层是真正的外科手术刀，第三层只有部分事件能阻断，前两层是建议。

三种状态，三种生命周期

仓库里的文件按生命周期可以分成三类，混在一起会读得很乱。

永久资产随仓库
走：`.claude/agents/`、`.claude/skills/`、`.claude/rules/`、`.claude/docs/templates/`、`studio-tests/`。这些是模板本体，克隆下来就带着。

项目状态随你的游戏走：`project.yaml` 描述引擎和模式；`design/` 放设计文档和注册表；`docs/architecture/` 放 ADR 和控制清单；`production/epics/` 放故事；`production/sprints/` 放迭代计划。这些一开始基本是空的或占位符。

会话状态随一次工作走：`production/session-state/active.md` 是检查点，`production/session-logs/` 是日志。这两处在 `.gitignore` 里，不进版本控制。

分类的意义在于知道什么可以改、什么不该改。修改 `.claude/skills/` 是定制模板，会被上游更新覆盖；修改 `production/epics/` 是推进项目；清理 `production/session-state/` 只影响当前会话的恢复能力。

目录与代码根

`CLAUDE.md` 导入的 `directory-structure.md` 给出一张表，说明各目录职责。其中最容易被忽略的一条是：代码根由引擎决定，`src/` 只是 Godot 那一行。

引擎	代码根	测试根
Godot	`src/`	`tests/unit/`、`tests/integration/`

引擎	代码根	测试根
Unity	`Assets/`	`Assets/Tests/EditMode/`、`Assets/Tests/PlayMode/`
Unreal	`Source/<Module>/`	`Source/<Module>/Private/Tests/`

这不是风格偏好。Unity 只编译 `Assets/` 和 `Packages/` 下的代码，UnrealBuildTool 只发现 `Source/` 下的模块。把代码写到 `src/` 在 Unity 项目里不会报错，它会安静地不参与编译。这个坑在第 14 章会展开，因为它解释了仓库里大量 `resolve_code_root` 逻辑的存在理由。

一次会话的加载顺序

把时序拉直，一次典型会话是这样开始的：

1. Claude Code 在项目根启动，读取 `CLAUDE.md`，展开其中的 `@` 导入，把 `directory-structure.md`、`coordination-rules.md`、`coding-standards.md` 和引擎版本引用一起装进上下文。
2. 读取 `.claude/settings.json`，注册 Hook 和权限规则，设置状态栏命令。
3. 触发 SessionStart 事件，`session-start.sh` 检查 `production/session-state/active.md`，如果有检查点就预览；`detect-gaps.sh` 检查项目缺什么。
4. 模型看到这些内容，等待你的第一条消息。

5. 你输入 `/dev-story`，Claude Code 找到 `.claude/skills/dev-story/SKILL.md`，执行正文开头那条被反引号包住的 shell 命令，把配置解析结果注入上下文，然后把正文交给模型。
6. 模型按正文的 Phase 1 到 Phase 7 执行，期间每次写文件都会经过 PreToolUse Hook。

这条链上只有第 5 步的 shell 注入是"先执行再看"，其余都是"先看再决定"。理解这一点，就理解了为什么技能正文里那些 `!` 开头的命令行如此重要：它们把配置求值的结果直接塞进提示词，不依赖模型自己去读文件、拼路径、猜默认值。

40 KB 的读保护

架构里有一个小设计值得单独说。`validate-read.sh` 挂在 PreToolUse 的 Read 事件上，只监视三类路径：`design/`、`docs/architecture/`、`production/session-state/`。当一次读取既没有 offset 也没有 limit，且文件超过 40 KB 时，脚本拒绝这次读取，`exit 2`，并在 stderr 里列出该文件的各级标题和行号。

这个设计的意图写在文件头：一次被拒绝的读取花掉一个回合，而它拦下的那次读取可能花掉几万 token。因为按范围读取总能通过，不会有工作被永久卡住。

它同时示范了这套架构的一种价值观：宁可让模型多走一步，也不要把整个大文件塞进上下文。`effects-map.md` 那份配置规范被反复提醒"不要在技能运行时读它"，理由是它的体积比 `CLAUDE.md` 每会话加载的全部内容还大若干倍。

常见误解

误解一：以为 `.claude/docs/` 下的文档都会自动加载。实际上只有 `CLAUDE.md` 里用 `@` 导入的才常驻，其余要按需读取，仓库还专门设计了读保护来防止误读。

误解二：以为 `settings.json` 里的 deny 列表就是全部安全边界。它只覆盖 Bash 和 PowerShell 的命令模式，写文件路径的检查在 Hook 里，而且多数只警告。

误解三：以为技能是脚本。技能正文是给模型看的步骤说明，可以被模型理解、跳过、改写；真正确定性的部分都在 `scripts/` 和 `hooks/` 的 Bash 里。

练习

1. 列出 `settings.json` 中注册的 Hook 事件，并指出哪些事件的退出码 2 会阻断操作。
2. 解释为什么 `validate-read.sh` 只监视三个目录，而不是全仓库。
3. 如果你的项目用 Unity，`CLAUDE.md` 里有一行导入 `@docs/engine-reference/godot/VERSION.md`，这行会造成什么问题？`/setup-engine` 会怎么处理它？

答案提示

1. 八个事件：SessionStart、PreToolUse、PostToolUse、Notification、PreCompact、Stop、SubagentStart、SubagentStop。能阻断的是 PreToolUse，Stop 和 SubagentStop 的退出码 2 会阻止停止，PreCompact 可以阻止压缩；其余事件退出码 2 只影响提示的去向。以官方 Hook 文档为准。
2. 三个目录是大型设计文档和状态文件的集中地，也是最容易一次读满上下文的地方。全仓库拦截会误伤代码阅读，成本高于收益。
3. 那行是注释标记的引擎引用导入，默认指向 Godot。Unity 项目每次会话都会加载无关的 Godot 版本说明。`/setup-engine` 会把它改写成 `@docs/engine-reference/unity/VERSION.md`。

03

入门：/start 如何把空白仓库变成项目

场景

克隆完仓库，你面对一个空项目：没有引擎，没有设计文档，只有一个模板和一堆规则。此刻最合理的动作不是随便挑一个技能，而是让仓库先看清自己处于什么状态，再决定路线。这就是 `/start` 的职责。

在固定快照上直接运行它依赖的状态脚本，输出如下（实测， `bash .claude/scripts/project-state.sh` ）：

```
root        /tmp/claude-game-studios-source   -
engine      -           unset
code_root   src         detected
source_files            0           src
concept     -           absent
systems_index           -           absent
gdd_files   0           design/gdd/*.md
adr         0           docs/architecture/adr-*.md
story       0           production/epics/**/story-*.md
prototype   0           prototypes/*/
sprint      0           production/sprints/*.md
```

```
milestone  0          production/milestones/*.md
origin_url https://github.com/Donchitos/Claude-Code-Game-Studios.git
.git/config
stage_yaml -          project.yaml
stage_txt  -          absent
stage      -          unset
phase      -          unset
phase_inferred        concept    nothing
```

这张表是 `/start` 的全部输入。每行三列：字段名、值、来源。第三列比第二列更重要，它告诉模型这个值是配置里写的、目录里推出来的，还是根本没找到。

状态判定

`/start` 的正文把这张表映射成判断：

- `engine` 为空且 `concept` 缺失，说明这是未配置的新项目。
- `source_files` 为 `unknown` 时要明确说"代码根无法解析"，绝不能说"没有代码"。这两句话的后续动作完全不同。
- `origin_url` 含 `donchitos/claude-code-game-studios` 时，说明你的 `origin` 还指向模板仓库，直接 push 会把提交送到上游模板。脚本会建议改名 `template` 再添加自己的远程。
- `phase_inferred` 只是按文件数量猜的，正文明确要求忽略它。一个只有代码、没有设计文档的项目不是 Production 阶段。

最后一条很值得注意。仓库自己提供了一个猜测字段，又在流程里禁止使用它。原因是猜测会把"代码很多"误判成"进入生产"，从而跳过设计和架构阶段。作者宁愿让你手动判定，也不愿让一个看似聪明的推断替代事实。

四条路径

`/start` 第二个问题只问一件事：你现在处于哪一步。

- A，完全没有想法。
- B，模糊的念头，比如"想做个太空题材"。
- C，清晰概念，知道类型和核心玩法，但没写下来。
- D，已有作品：设计文档、原型、代码或计划。

选完之后，它只给出一个"下一步"，不给完整路线。理由是路线长度取决于 rigor，而 rigor 还没确定。选 D 时会先报告扫描结果，比如"代码根下有 N 个源文件，N 份设计文档，引擎未设置"，再决定从 `/project-stage-detect` 还是 `/setup-engine` 开始。

如果回答与扫描结果冲突，正文要求指出一次并重新问。比如你选了 A"没有想法"，但磁盘上有代码，它会说"我在 `src/` 发现了代码，你是不是想选 D"。

强度推断

第四个阶段是确定 rigor。规则是一张三行的信号表：

你的描述里出现	推荐 rigor
开放世界、RPG、模拟、沙盒、派系、制作、经济、MMO、多人、很多系统	full
发布、商业、Steam、一个清晰的核心循环	standard
Game Jam、周末、原型、第一个游戏、小、只是试试	minimal
以上都没有，或还没想法	minimal（默认）

推断结果会以卡片形式展示，并说明理由用的是你的原话。卡片上还会写明后果：minimal 大约四步到代码，standard 大约八份设计文档，full 在 standard 基础上加验证构建和全量证据。

关键在于：`/start` 不会问你"要不要 rigor"，它只问"是否保存"。rigor 的调整入口统一在 `/settings`。这个设计避免了六个相关设置被逐个询问，也避免了用户在没有上下文时凭空选一个抽象档位。

写入与不写入

`/start` 只写三项：`project.stage`（以及旧版镜像 `production/stage.txt`）、`modes.rigor`、`modes.automation`。写入必须经过显式确认，并且通过 `set-stage.sh` 和 `config-set.sh` 两个辅助脚本完成。

它明确不写 `modes.review_mode`，也不写另外五个被 rigor 前置的设置。原因写在 `project.yaml` 的注释里：一旦某个键被显式写入，rigor 展开就再也改不动它，用户之后调用 `/settings` 调档位会失效。

这是一个容易犯的错误。很多人会想"我先把默认值都写进配置文件，清楚一点"。在这套系统里，这样做等于把档位锁死。

交接而不是代跑

最后一个阶段只有一行输出：`Type /setup-engine to begin.`

正文特别要求：不要替用户启动那个技能，即使用户刚说了"好，开始吧"。用户必须自己输入命令。这个细节看起来吹毛求疵，实际是权限设计的一部分。用户亲手输入命令，意味着他知道接下来会发生什么；模型代为调用，则把一个需要确认的动作变成了自动连锁。

常见误解

误解一：`/start` 会自动创建完整的项目骨架。它只写三个设置，然后交接。真正的引擎配置在 `/setup-engine`，设计在 `/brainstorm`。

误解二：`code_root: src` 说明引擎是 Godot。这里 `src` 只是"目录里恰好只有一个候选根"的推断结果，来源列写的是 `detected`，不是 `engine.name`。

误解三：rigor 推断是随意的。它有一张信号表，而且推断结果会在卡片上给出理由，用户可以当场否决。

练习

1. 在你自己的项目副本上运行 `project-state.sh`，找出三处"来源是推断而不是配置"的字段。
2. 假设你选了路径 C（清晰概念）并输入"一个关于送信的叙事小游戏"，按信号表推断 rigor，并说明理由。
3. 如果 `/start` 把 `modes.rigor` 写进 `project.yaml`，之后你手动把 `modes.workflow` 也写进去，会发生什么？

答案提示

1. 常见的是 `code_root`（来源 `detected`）、`phase_inferred`（来源 `nothing` 或文件计数）、`gate_level` 的默认部分。任何来源列不是 `project.yaml` 或 `project.local.yaml` 的值都属于推断或默认。
2. 叙事、小游戏、没有出现商业发布或大型系统信号，落在 minimal 或 standard 边界。按表，"一个清晰的核心循环"倾向 standard，但"小"倾向 minimal，卡片应展示推断并给出理由，由用户确认。
3. `modes.workflow` 被显式钉住，rigor 展开不再改变它。之后调 rigor 只影响其余五个设置，workflow 保持你写的值。这是 `project.yaml` 注释里警告的行为。

04

配置求值：一个 rigor 如何长出六个设置

场景

你在 `project.yaml` 里只写了一行：

```
modes:
  rigor: standard
```

然后在技能里看到 `workflow: standard`、`qa.level: standard`、`review_mode: lean`、`team.size: individual`、`docs.density: balanced`、`story_granularity: balanced`。六个值你一个都没写。

到 `project.local.yaml` 里写 `modes.review_mode: solo`，自己的会话立刻从 lean 切到 solo，而不影响团队里其他人。再删掉 `project.yaml` 里的 rigor，所有值回落到 minimal 展开。

这套行为集中在 `.claude/hooks/yaml-helper.sh` 的 `resolve_setting` 函数里。它是全仓库被调用次数最多的逻辑，理解它就理解了技能的"环境"。

解析链

`resolve_setting` 按固定顺序查五个来源，第一个给出合法值的就是答案：

1. `project.local.yaml`，仅当该键在"本地可覆盖"白名单里。
2. `project.yaml`。
3. 旧版纯文本镜像，例如 `production/review-mode.txt`、`production/stage.txt`。
4. rigor 展开。
5. 文档化的末端默认值。

每一级都有两个动作：读到值后先做枚举校验，非法值丢弃并继续往下走，而不是让一个拼错的值污染所有技能。最终输出两列，值和来源。

在快照仓库上直接调用，输出是：

```
=== CCGS Resolved Config ===
rigor: minimal (default)
review_mode: solo (rigor:minimal)
automation: collaborative (default)
workflow: minimal (rigor:minimal)
docs.density: terse (rigor:minimal)
story_granularity: coarse (rigor:minimal)
qa.level: minimal (rigor:minimal)
team.size: individual (rigor:minimal)
project.stage: not set (unset)
```

```
automation_always_ask: scope_changes, file_deletions, schema_changes
(default)
engine: unset (fall back to .claude/docs/technical-preferences.md)
testing.strict: logic=unset integration=unset visual=unset ui=unset
config=unset (unset = each skill applies its own default)
performance.enforce: warn (default)
platform.cert_tier: (unset -- ask which platforms are in scope)
system_overrides: none
notes: none
```

括号里的来源比数值有用。`rigor: minimal (default)` 说明项目没配过，`review_mode: solo (rigor:minimal)` 说明它来自展开而不是显式设置。排查"为什么技能行为不对"时，先看来源。

展开表

`yaml-helper.sh` 里有一张三行表，把 rigor 翻译成六个键：

```
minimal :: modes.workflow=minimal, docs.density=terse,
qa.level=minimal,
           modes.story_granularity=coarse, modes.review_mode=solo,
team.size=individual
standard:: modes.workflow=standard, docs.density=balanced,
qa.level=standard,
           modes.story_granularity=balanced, modes.review_mode=lean,
team.size=individual
```

```
full    :: modes.workflow=full, docs.density=thorough, qa.level=full,
           modes.story_granularity=fine, modes.review_mode=full,
team.size=studio
```

展开位置很关键：它在所有显式来源之后、末端默认之前。因此显式写了 `docs.density: terse` 的项目，行为与引入 rigor 之前完全一致。这也是向后兼容的保证。

`modes.workflow` 有点特殊。它有一个按系统覆盖的机制 `workflow_overrides.system_overrides`，可以只把战斗系统提到 full，其余保持 minimal。解析时先看系统覆盖，再看 `modes.workflow`，最后才是 rigor 展开。

为什么要一条链，而不是一个配置文件

设想三种真实需求。

团队共享的规范：引擎、命名、性能预算。这属于项目，应该进版本控制。`project.yaml` 承担这一层。

个人偏好：我习惯 solo 评审，我的机器上下文小，不想每次等四个总监。这属于开发者，不该污染团队配置。`project.local.yaml` 承担这一层，同时被 gitignore。

历史遗留：旧版本把评审模式写在 `production/review-mode.txt`，把阶段写在 `production/stage.txt`。已经升级的项目里这些文件还在。镜像文件承担这一层，但优先级低于 YAML。

三种需求冲突时，顺序就是答案：个人高于项目，项目高于遗留，遗留高于展开，展开高于默认。

读白名单与写白名单必须同步

`yaml-helper.sh` 里有一段注释值得抄进任何配置系统：

> 这个列表必须等于 `/settings --local` 的写入白名单。如果解析时查询 `project.local.yaml` 的范围比 `/settings` 能写的范围窄，那么一次 `/settings --local modes.review_mode=solo` 会成功写入、成功显示，然后被每个技能忽略。

这不是理论问题，是"写列表和读列表不一致"这类 bug 的典型症状：配置看起来生效了，实际没有。仓库的应对方式是把两个列表放在同一处，并在测试里断言它们同步。

当前可本地覆盖的是 `modes.review_mode` 和 `team.size`。理由是这两个是个人工作习惯，其余设置影响团队产物的一致性。

失败模式

文档列了一张失败清单，每一条都对应一种真实的坏输入：

- `project.yaml` 不存在：走遗留文件和默认，备注 `project.yaml absent`。

- 文件存在但为空：备注 `project.yaml is EMPTY`，与不存在区分开。被截断的写入会表现成这个状态。
- YAML 格式错误：解析失败的键留空，继续往下走。
- 用了 Tab 缩进：拒绝，并在备注里给出**行号**。
- CRLF 换行：正常解析，照顾 Windows 默认。
- UTF-8 BOM：正常解析，BOM 不会污染第一个键。
- 键存在但值为空：视为未设置，继续往下，不会解析成空字符串。
- 枚举值非法：继续往下，并在备注里点名被拒绝的值。
- 重复键或重复块：块合并，重复键取最后一个值，并在备注里列出两处行号。

最后一条值得单独说。多数 YAML 工具遇到重复块只保留最后一个，这里选择合并，因为两个 `modes:` 块同时出现时，用户通常是想补充而不是覆盖。合并比丢数据安全。

还有一类输入不产生备注：形状错误。比如 `rigor:` 下面跟了一个列表而不是标量，键会解析为空，回落到默认，且不会告诉你。文档明确承认这个盲区，并给出排查方法：当某个设置看起来被忽略时，检查它的键是不是一行一个标量。

技能里怎么用

技能正文开头通常有一行：

```
!`bash "${CLAUDE_SKILL_DIR}/../../hooks/yaml-helper.sh"
resolve_config --keys
automation,workflow,qa.level,gate_level,engine_values,specialists,per
f_budgets,system_overrides`
```

`!` 前缀让 Claude Code 在把正文交给模型之前执行这条命令，把输出注入上下文。之后的正文写着"Resolved above, use as-is"。模型不需要自己读 YAML、拼路径、处理优先级。

这条设计把"确定性计算"从模型手里拿走了。代价是 `allowed-tools` 必须精确授予这条命令，否则整次技能调用会因为缺少授权而中止。仓库在 `config-resolution.md` 里专门写了授权格式，并区分了"预批准"和"准入控制"：`allowed-tools` 只是免去权限提示，并不限制其他命令；而未授权的注入命令会让技能直接失败。

常见误解

误解一：`project.yaml` 写什么就生效什么。旧版镜像文件、非法枚举、本地覆盖都可能改变结果。要用 `resolve_config` 看最终值和来源。

误解二：`rigor` 是总开关，改它一定全变。被显式钉住的键不受影响，这正是那六个键的意义。

误解三：`project.local.yaml` 能覆盖任何设置。只有白名单里的两个键可以，其余会被忽略并在备注里点名。

练习

1. 在副本里把 `modes.rigor` 设为 `full`，运行 `resolve_config`，记录六个派生值。
2. 再把 `modes.workflow` 显式设为 `minimal`，重新运行，观察哪些值变了，哪些没变。
3. 构造一个 Tab 缩进的 `project.yaml`，看备注如何报告行号；再构造一个 `rigor` 下面跟列表的文件，解释为什么没有备注。

答案提示

1. 应得到 workflow=full、docs.density=thorough、qa.level=full、story_granularity=fine、review_mode=full、team.size=studio，来源全部标 `rigor:full`。
2. workflow 固定为 minimal（来源 `project.yaml`），其余五个仍从 rigor 展开，来源 `rigor:full`。
3. Tab 缩进会得到带行号的备注；列表形状会被解析为空值并回落到默认，脚本无法区分"故意的空值"和"写错了形状"，因此不报备注。排查时手动检查键的形状。

05

角色系统：49 个 Agent 如何分工

场景

你让 `/dev-story` 实现"战斗中的格挡判定"。它在 Phase 4 查一张表，发现这是 Core 层的玩法逻辑，于是选中 `gameplay-programmer`。这个角色有自己的提示词、自己的工具清单、自己的回合上限。它读到故事文件、需求条目、ADR 摘要，然后开始写代码。

与此同时，`technical-director` 可能出现在评审环节，`qa-tester` 出现在测试环节，`godot-shader-specialist` 出现在涉及着色器的故事里。它们不是常驻进程，是按需派生的子代理。

角色的物理形态

每个角色是一个 Markdown 文件，放在 `.claude/agents/` 下，文件名就是角色名。开头是 YAML 前置元数据：

```
---
name: gameplay-programmer
description: "Implements designed mechanics as code ..."
tools: Read, Glob, Grep, Write, Edit, Bash
```

```
model: inherit
maxTurns: 20
---
```

`name` 和 `description` 决定它能被谁、在什么条件下调用。`tools` 是它能用的工具白名单。`model` 是模型档位。`maxTurns` 是回合上限，防止一个子代理无限循环。

在固定快照上统计 49 个角色文件：

- 模型分布：`inherit` 30 个，`sonnet` 16 个，`opus` 3 个。
- 回合上限：全部 49 个都设置了，范围从 10 到 30，多数是 20。
- 工具组合只有 5 种变体：最宽的 28 个角色拥有 `Read, Glob, Grep, Write, Edit, Bash`；9 个只有读写不加 Bash；6 个额外有 `WebSearch`；3 个有 `Agent` 可以再派生下级。

这三个 `opus` 角色是 `creative-director`、`producer`、`technical-director`，对应 README 里的第一层总监。

三层结构

README 把角色分成三层，模仿真实工作室：

第一层，总监：创意总监守愿景，技术总监守架构，制作人守进度和范围。

第二层，部门主管：游戏设计、主程序、美术、音频、叙事、QA、发布、本地化。

第三层，专家：玩法、引擎、AI、网络、工具、UI 程序员，系统、关卡、经济设计师，技术美术、音效、编剧、世界构建、UX、原型、性能、运维、分析、安全、QA、无障碍、Live Ops、社区。

三层之上还有一套协调规则，写在 `.claude/docs/coordination-rules.md`：

1. 垂直委派。总监交给主管，主管交给专家，复杂决策不跳级。
2. 水平咨询。同层可以互相咨询，但不能替对方做决定。
3. 冲突升级。两人意见不合时交给共同上级；没有共同上级时，设计冲突找创意总监，技术冲突找技术总监。
4. 变更传播。跨领域的改动由制作人协调。
5. 不做单方面跨域修改。没有明确委派，不得改自己目录之外的文件。
6. 按章节读设计文档。任务说明里点名需要的章节，用 Grep 找标题，再按行号读范围；整篇读 40 KB 以上的文档会被 Hook 拒绝。
7. 不往上游提交。即使是引擎或插件的 bug，也只起草报告，由用户自己提交。

第七条是一个有意思的伦理约定。多代理系统很容易被要求"顺手帮我给上游提个 issue"，而这类操作会以用户身份对外发声。仓库选择只起草不提交。

协作协议

每个实现类角色的提示词里都有一段几乎相同的"协作协议"，来自 `COLLABORATIVE-DESIGN-PRINCIPLE.md` 的落地版本。以 `gameplay-programmer` 为例，写代码之前的顺序是：

1. 读设计文档，区分明确的和含糊的部分，标记实现难点。
2. 问架构问题："这里应该是静态工具类还是场景节点？"，"这个数据该放哪？"，"设计文档没写这个边界情况，应该怎么处理？"
3. 先提架构方案：类结构、文件组织、数据流，说明为什么这样选，指出权衡。
4. 实现时保持透明：遇到规格含糊就停下来问，不猜。
5. 写文件前获得批准："我可以写入 [路径] 吗？"
6. 主动提供下一步选项。

这段协议的核心是把模型从"生成器"改成"顾问"。它不禁止模型写代码，但要求写之前有明确的批准动作。

一个有边界的例外

如果严格照字面执行，"每次写文件都要问"会让多代理流水线卡死。父代理派生五个子代理，每个子代理写完都要回来问用户，用户被五次询问轰炸。

仓库给了一个精确的例外，写在角色提示词里：如果子代理是被某个编排器派生的，而那个编排器的提示词**点名了产物的目标路径**，子代理可以直接写入，不再单独询问。限制条件有四条：

- 仅限 `production/` 、 `docs/` 、 `tests/` 下的新产物。
- 不能是对已有源码或配置的修改。
- 路径必须是编排器给的，不能是子代理自己选的。
- 直接被调用、或没有被指定路径时，仍然要问。

这是一个典型的"用范围换流畅"的设计。批准的单位从"每个文件"变成"每个阶段"，而阶段是用户在流水线里明确确认过的。

模型档位不等于计费策略

`model-tiers.md` 花了大量篇幅解释一个容易误解的点：技能的 `model:` 和角色的 `model:` 是两套机制。

技能档位只在一种情况下生效：你亲手输入 `/skill-name` ，且当前不在 auto 模式，或者档位是 sonnet/opus。如果模型自己通过 Skill 工具启动了那个技能，档位不生效，用会话模型。文档因此提醒：不要在 Sonnet 会话里告诉用户"用 haiku 技能省钱"，因为如果技能是被模型启动的，根本没有切换。

角色档位是另一回事：子代理是独立会话， `model:` 会被应用。这也是 30 个角色写 `inherit` 的原因，它们跟着会话模型走，不额外指定。

文档还提到一个细节：Opus 会话里跑一个 `opus` 钉住的角色，会用会话的同一个模型版本，包括 `[1m]` 长上下文变体，不会拿到比父会话更小的窗口。

角色规则之间的张力

静态阅读会发现一处不一致。`gameplay-programmer` 的职责里写着"为所有玩法逻辑写单元测试"，"不得跳过玩法逻辑的单元测试"。但 `/dev-story` 在 `qa.level: minimal` 时明确指示："不要为这个故事写测试文件，测试证据在 minimal 级别豁免。"

两者会在 minimal 档位下同时进入上下文。角色提示词说必须写，技能流程说不要写。

这不是致命 bug，因为技能流程更具体、更靠后，模型通常会服从具体指令。但它是一个真实的规则冲突，值得在读的时候记下来。任何多角色系统都会遇到这类问题：通用角色规范与具体流程分支的优先级没有形式化定义。仓库没有给角色规则标优先级，靠的是提示词里的"更具体者优先"这一隐含约定。

如果你要改造这套系统，一个低成本的修法是在角色提示词里加一句"流程级指令优先于本文件的通用要求"，或者让技能在派生角色时显式传递当前档位。

常见误解

误解一：49 个角色会同时参与一次任务。实际上一次 `/dev-story` 通常只派生一到两个角色，团队流水线也会按 `team.size` 折叠。

误解二：角色是独立进程。它们是同一会话内的子代理，共享权限上下文，结果返回父会话。

误解三：`tools` 字段是限制。它决定子代理能用哪些工具，写文件仍然受父会话的 Hook 和权限约束。

练习

1. 统计 `.claude/agents/` 下各模型档位的数量，并解释为什么大多数是 `inherit`。
2. 找出三个拥有 `Agent` 工具的角色，推测它们为什么需要派生下级。
3. 阅读 `gameplay-programmer` 和 `dev-story` 关于测试的要求，写出你认为合理的优先级规则，并说明如何在提示词层面落地。

答案提示

1. 30 个 inherit、16 个 sonnet、3 个 opus。inherit 让角色跟随会话模型，避免为每个角色维护档位，也避免用户在 Sonnet 会话里被动升到 Opus。
2. 通常是编排型或主管型角色，需要把工作拆分给专家。可以用 `grep -l "Agent"` 或读前置元数据确认。

3. 一种可行规则：具体流程的显式指令 > 角色通用要求 > 团队默认。落地方式是在角色文件顶部声明优先级，或在派生子代理的提示词里写明"当前 qa.level 为 minimal，测试豁免"。

06

Skill 协议：一个命令从解析到交接

场景

你输入 `/dev-story production/epics/stealth/story-003.md` 。屏幕上先出现一行配置解析结果，然后模型开始按 Phase 1 到 Phase 7 推进：找故事、检查前置条件、标记进行中、选择程序员、派活、检查结果、写总结。

这一章拆开这条命令的执行过程。理解它之后，你能自己写一个符合这套协议的技能，也能判断一个技能在什么情况下会失败。

文件结构与前置元数据

`.claude/skills/` 下有 74 个目录，每个目录一个 `SKILL.md` 。以 `dev-story` 为例：

```
---
name: dev-story
description: "Implements one story in code with tests, following its
ADRs. ..."
argument-hint: "[story-path]"
user-invocable: true
```

```
disable-model-invocation: true
allowed-tools: Read, Glob, Grep, Write, Edit, Agent, AskUserQuestion,
  Bash(bash "*/.claude/skills/dev-story/../../hooks/yaml-helper.sh"
resolve_config *),
  Bash(bash .claude/scripts/story-facts.sh *), ...
model: sonnet
---
```

每个字段都有明确作用：

`name` 是命令名，也是模型通过 Skill 工具调用时用的标识。`description` 决定什么时候该用它，写得越具体，误触发的机会越小。`argument-hint` 告诉用户该带什么参数。`user-invocable: true` 表示可以手动输入 `/dev-story`。`disable-model-invocation: true` 表示模型不能自己启动它，必须由用户触发，这对会产生文件变更的技能很常见。

`allowed-tools` 是一份精确到子命令的授权清单。注意它授权的是特定 Bash 调用，而不是整个 Bash 工具。`Bash(bash .claude/scripts/story-facts.sh *)` 只允许跑这个脚本，不允许跑 `rm`。

`model: sonnet` 是该技能建议的模型档位，是否生效取决于启动方式，第 5 章已经解释。

正文的第一段是命令

几乎每个技能的正文开头都有一行被反引号包裹的 shell 命令，前面带 `!`：

```
!`bash "${CLAUDE_SKILL_DIR}/../../hooks/yaml-helper.sh"
resolve_config --keys
automation,workflow,qa.level,gate_level,engine_values,specialists,per
f_budgets,system_overrides`
```

Claude Code 在加载技能时执行它，把标准输出直接插入提示词。紧接着一行：

> Resolved above — use as-is. No block → defaults in `.claude/docs/config-resolution.md`.

这两句合起来完成了一次"确定性注入"。模型不需要知道配置在哪、怎么解析、有哪些层级，它只需要读注入结果。与此同时，注入失败的情况也有兜底说明。

这个模式有一个前提：命令必须被精确授权。`allowed-tools` 里的那段长模式就是为此存在。授权写错，整次技能调用中止，而不是降级运行。仓库在 `config-resolution.md` 里说明了这个取舍：注入是严格的，权限提示是宽松的，两者不能混用。

七段式流程

`dev-story` 的正文是一张清单加七个阶段：

```
- [ ] Phase 1: story found, tier resolved
- [ ] Phase 2: prerequisites checked (ADR, manifest, dependencies)
- [ ] Phase 3: story marked In Progress (one ask)
- [ ] Phase 4: programmer and engine specialist chosen
- [ ] Phase 5: work handed out with a full brief
- [ ] Phase 6: finished? parse check, run and look, one Run result
line
- [ ] Phase 7: summary, ## Files, checkpoint, closing block
```

清单的作用不只是给模型看，也是给用户看进度。每个阶段有明确的输入、输出和停止条件。

Phase 1 解析参数。没有参数时读会话检查点里的当前任务，确认后继续；没有故事时给出下一步命令，而不是猜一个。

Phase 2 做前置检查，按 workflow 档位分三列。full 档要求需求注册表存在，缺失就停止；standard 档只是可选；ADR 必须是 Accepted 状态；依赖故事必须是 Complete。任何一项不满足，就进入 Phase 7 写 BLOCKED 检查点，不派生程序员代理。

Phase 3 只问一次："我可以把故事标记为进行中吗？这会改故事状态、更新冲刺状态文件，并在结束时列出改动文件。"用户拒绝，就继续实施但不改状态，并在总结里写明"Story not marked In Progress — declined"。

Phase 4 用一张表选人：Foundation 层给引擎程序员，UI 类型给 UI 程序员，Visual/Feel 给玩法程序员，AI 给 AI 程序员，网络给网络程序员。配置故事不需要代理，直接改数据文件。

Phase 5 派活。这里有一条重要约束：简报要点名文件，不粘贴整份文档。故事文件、需求条目、ADR 摘要、控制清单的对应层规则、命名规范、性能预算、测试路径，逐项列出。明确告诉子代理"哪些输入在当前离线不存在，不要去找"。

Phase 6 检查结果。先看文件是否真的写出来了，再看测试是否存在，然后做解析检查，最后运行并观察。三个结果词被固定下来：`OBSERVED`、`NOT VERIFIED`、`N/A`。

Phase 7 输出固定格式的总结，更新故事的 `## Files`，写会话检查点，最后打印三行收尾：故事状态、运行结果、下一步。

提问协议

技能里反复出现同一段话：

> 用提问工具询问。没有提问工具时，把选项编号列出并结束回合等待。被跳过或关闭的问题不是回答，用纯文本再问一次。

这段话解决了一个真实故障：在无人值守或工具缺失的环境里，提问调用可能返回"全部跳过"。如果模型把跳过当成默认同意，就会在没有批准的情况下写文件。仓库的处理是明确区分"没回答"和"同意"，遇到前者就停

下。

`automation-modes.md` 把这个规则扩展到三种自动化模式：collaborative 每次都问；guided 只问重大决策；autonomous 不问但要把决策写进日志。三种模式下，"被跳过的问题不是回答"都成立。

引用而不是内嵌

技能正文经常写"读 `.claude/docs/automation-modes.md` "或"见 `.claude/docs/director-gates.md` "，而不是把整份规范抄进来。

好处是单一事实来源，改一处所有技能同步。代价是模型可能不读，或者读了不照做。仓库用两种方式缓解：关键规则在技能里重述一两句，其余交给按需加载。

在 `director-gates.md` 里还有一条反向约束：技能派生总监做评审时，**不要自己读评审定义文件**，而是把文件路径放进子代理的提示词，让子代理在自己的上下文里读。理由是节省父会话的 token，同时保证评审者看到完整定义。

收尾契约

好的技能有明确的结束条件。 `dev-story` 的最后一段规定：

```
Story: [path] — [Implemented | INCOMPLETE — <breakage> | BLOCKED —
<reason>]
Run result: [OBSERVED — … | NOT VERIFIED — … | N/A — …]
```

```
Next step: [...]
```

状态词只有三种，每种对应不同下一步。`INCOMPLETE` 时下一步是继续实施，`BLOCKED` 时下一步是解除阻塞的具体动作。没有"基本完成"这种模糊状态。

`story-done` 有类似的四态判决：COMPLETE、COMPLETE WITH NOTES、NOT ASSESSED、BLOCKED，并规定了优先级顺序。第 9 章会拆解。

常见误解

误解一：技能是确定性的。它是一份给模型的步骤说明，模型可能跳步、合并步骤、或者误解某条规则。真正的确定性在脚本里。

误解二：`allowed-tools` 限制了技能能做什么。它只是免去权限提示，不构成沙箱；真正能拦住的是 `settings.json` 的 deny 列表和 Hook 的退出码。

误解三：技能越详细越好。`effects-map.md` 被反复提醒不要在运行时读，因为它太大。技能正文也有同样的问题，过长会挤占上下文并降低遵循率。

练习

1. 找两个技能，对比它们的 `allowed-tools`，说明为什么一个需要 `Agent` 而另一个不需要。

2. 阅读 `dev-story` 的 Phase 2，列出 full 档和 minimal 档各自的前置条件差异。
3. 为一个"生成变更日志"的技能设计前置元数据和收尾契约，要求它在没有 git 历史时明确报告而不是编造。

答案提示

1. 需要派生专家的技能（`dev-story`、`team-*`）需要 `Agent`；只做读写分析的技能（`code-review`、`design-review`）不需要。授权越窄，越不容易误用。
2. full 档要求需求注册表存在、ADR 必须 Accepted、控制清单缺失要警告；minimal 档不期待需求注册表，ADR 只在故事点名时检查。
3. `allowed-tools` 含 `Bash(git log *)`；收尾契约里明确 `NO_HISTORY` 状态和"未生成，因为仓库没有提交记录"的输出，禁止用推测填充。

07

产物模型：故事、需求、ADR 三张网

场景

`/dev-story` 选中一个故事，读了它的验收标准，开始写代码。半小时后它报告完成。`/story-done` 接手，逐条核对验收标准，发现第三条"格挡成功时播放火花特效"既没有截图，也没有运行记录，于是标成 UNTESTED，判决停在 NOT ASSESSED。

这两个技能之所以能对同一份内容做出可比较的判断，是因为它们读的是同一种产物：故事文件。这一章讲清三类核心产物如何互相咬合。

故事：最小可执行单元

`.claude/docs/templates/story.md` 定义了故事的全部字段。头部是元数据：

```
> **Epic**: [epic name]
> **Status**: [Ready | Blocked — BLOCKED: <reason>]
> **Layer**: [Foundation / Core / Feature / Presentation]
> **Type**: [Logic | Integration | Visual/Feel | UI | Config/Data]
```

```
> **Estimate**: [S | M | L]
> **Manifest Version**: [date from control-manifest.md header]
> **Last Updated**: [set by /dev-story]
```

`Layer` 决定派给哪个程序员：Foundation 给引擎程序员，Core 和 Feature 按类型分，Presentation 给 UI 或玩法。`Type` 决定需要什么证据：Logic 要单元测试，Integration 要集成测试或试玩记录，Visual/Feel 要截图加签字，UI 要截图，Config/Data 要冒烟报告。

这两个字段合起来就是"谁做、怎么算做完"。

正文分八节：

- Context：指向 GDD 文件、需求 ID、ADR、引擎与风险、控制清单规则。
- Acceptance Criteria：从 GDD 摘出的、属于这个故事的验收条件。
- Implementation Notes：从 ADR 的实施指南提炼的具体指示，程序员读这一段而不是读整份 ADR。
- Out of Scope：由邻近故事负责的内容，明确不在此实现。
- Files：计划改动的代码和测试文件，`/dev-story` 结束时替换成实际清单。
- QA Test Cases：QA 主管写的测试用例，开发者照着实现，不得自行发明。
- Test Evidence：所需证据的类型和路径。
- Dependencies：依赖哪些故事，解锁哪些故事。

有一句注释很关键：需求文本的权威位置是 `docs/architecture/tr-registry.yaml`，故事里引用的版本可能在评审时已经过时，所以"review time read fresh"。

需求注册表：事实的唯一出处

`docs/architecture/tr-registry.yaml` 存的是一条条编号需求，形如 `TR-combat-001`。故事引用 ID，评审时按 ID 回表读最新文本。

这解决了一个常见问题：故事创建时抄了一份需求描述，三周后设计改了，故事里的文字变成了化石。用 ID 引用，文本只有一份。

同一个模式出现在 `design/registry/entities.yaml`，存的是跨文档的游戏世界事实：实体、物品、公式、常量。文件头规定了写入规则：

- 只登记跨系统边界的事实，单份 GDD 内部使用的公式不必登记。
- 永不删除条目，用 `status: deprecated` 标记。
- 值改变时更新值、写修改日期，并注释旧值和哪份 GDD 改的。
- `source` 是权威 GDD，其他引用它的文档列在 `referenced_by`。
- 新 GDD 引用已有条目时，追加路径，不新建重复条目。

文件顶部还列了读取方和搜索模式，比如"所有实体名"用 `^ - name:`，某个 GDD 拥有什么用 `source: design/gdd/combat.md`，某个 GDD 引用了什么用路径加 `-B 8`。

这些搜索模式不是给模型自由发挥的，而是给 `.claude/scripts/registry.sh` 的固定读法。第 11 章会展开。

ADR：决策的时间戳

架构决策记录（ADR）放在 `docs/architecture/`，编号 `ADR-NNNN`。故事里引用三个字段：

- `ADR Governing Implementation`：哪一份 ADR 管这个实现。
- `ADR Decision Summary`：一两句话的决策摘要。
- `ADR Version`：ADR 的 `Last Verified` 日期，没有就用 `Date`，都没有就写 `unversioned`。

`/dev-story` 的规则是：版本相同、故事没有版本字段、或双方都是 `unversioned` 时，信任故事的摘要；版本不同时，才去读 ADR 并处理版本冲突。

注意这里的工作量分配：程序员默认不读整份 ADR，只读故事里的摘要。这样省上下文，代价是摘要可能失真。仓库用版本号来检测失真，而不是每次重读。

ADR 的状态也参与门禁。`Proposed` 状态的 ADR 不能作为实施依据，必须先用 `/architecture-decision accept ADR-NNNN` 转为 Accepted。`Deprecated` 或 `Superseded` 状态会要求把故事指向继任者。

控制清单：跨故事的架构约束

`docs/architecture/control-manifest.md` 是一份按层组织的规则清单，每层列出必须遵守的模式、禁止的模式、性能护栏。故事头部记录它创建时的清单版本。

版本不一致时，`/dev-story` 会读一个版本冲突处理文件；`/story-done` 则把"故事写在旧版清单上"记为 ADVISORY，建议跑 `/story-readiness` 检查。

这是三张网里最容易被忽略的一张。它管的是"这个架构里不允许出现什么"，比如某个层不能用直接引用，必须走事件总线。单看一个故事不会发现违规，只有对照清单才能判断。

三张网如何咬合

把三类产物放在一起看，关系是：

```
GDD（设计事实）
  ├─ 需求注册表（可引用的需求 ID）
  ├─ ADR（决策 + 版本）
  │     └─ 控制清单（按层的约束）
  └─ 故事（验收标准 + 证据要求 + 文件范围）
        └─ 冲刺计划（排序与容量）
```

故事是最小的执行单元，向上引用事实，向下产生证据。评审技能不需要重读全部设计，只需要顺着故事里的引用回表。

这个结构的代价是前期投入。standard 档位下，写第一行代码前会有 30 份文档。作者自己的对照实验显示，这些文档带来的是可追溯性，而不是更好的游戏。理解这一点，你就知道该在什么规模的项目上用哪一档。

状态机

故事状态没有单独的状态机文件，它由几个约束共同维护：

- 模板允许的初始状态是 `Ready` 或 `Blocked`。
- `/dev-story` 在开始时把状态设为 `In Progress`，`Last Updated` 写今天。
- `/story-done` 只在用户明确选择关闭时把状态设为 `Complete`，并追加完成记录。
- `production/sprint-status.yaml` 同步维护 `status: in-progress` / `done` 和顶层 `updated` 日期。

没有任何脚本强制这些转换。一个故事可以从 Ready 直接跳到 Complete，只要有人手动改。状态机的强制力来自技能流程，而不是代码。

常见误解

误解一：故事文件是任务的完整规格。它是一份索引加验收标准，权威事实在设计文档和注册表里。

误解二：ADR 每次实施都要读。默认读故事里的摘要，版本不一致时才回原文。

误解三：注册表可以随便删条目。文件头明确禁止删除，只能标废弃。因为历史故事和评审记录会引用旧 ID。

练习

1. 打开故事模板，找出三个直接决定"派给谁"和"需要什么证据"的字段。
2. 假设一个需求文本在故事创建后被修改，说明评审时会读到哪个版本，依据是什么。
3. 为一个"设置菜单"功能写一个最小故事头部，标注 Layer、Type 和所需证据。

答案提示

1. `Layer` 决定程序员角色，`Type` 决定证据类型，`Status` 决定能否开始。`Manifest Version` 影响版本冲突检查。
2. 评审读注册表里的最新文本，因为故事只存 ID，并且模板注释要求评审时回表读取。
3. 例如 Layer: Presentation，Type: UI，证据是一张或多张截图放在 `production/qa/evidence/<story-slug>/`，不需要单元测试。

08

实施链路：/dev-story 怎么把故事变成代码

场景

故事已经就绪：`story-003-melee-parry.md`，Core 层，Logic 类型，ADR 已接受，依赖故事全部完成。你输入 `/dev-story production/epics/stealth/story-003-melee-parry.md`。

接下来发生的事情可以分成两条线：一条是文本处理线，决定模型看到什么；一条是实际执行线，决定磁盘上多出什么文件。这一章顺着两条线走一遍。

第一步：把故事变成事实表

技能调用 `story-facts.sh <story-path>`，拿到状态、类型、层、需求 ID、ADR、验收条件数量、测试证据状态、依赖。脚本只输出观察，不输出判决。

在没有故事的模板仓库上运行 `story-facts.sh --all`，输出是一张空表：

```
stories    0          production/epics/**/story-*.md
complete   0
in_review  0
in_progress           0
todo       0
blocked    0
other      0
no_status  0
```

`story-status.sh` 更短，只有一行 `STORIES none`。这两个脚本的价值在于：当模型问"有哪些故事可以做"时，答案来自磁盘，而不是来自记忆。

第二步：解析工作档位

技能注入的配置里有 `workflow`、`qa.level`、`gate_level`、`specialists`、`perf_budgets`、`system_overrides`。工作档位不是全局 rigor，而是按故事所属系统解析的：故事里的 `GDD:` 路径给出系统名，先查该系统的覆盖，再查项目值。

所以一个项目可以整体 minimal，只把战斗系统提到 full。此时战斗故事的 Phase 2 会要求需求注册表和 Accepted ADR，其他故事不受影响。

第三步：检查是否允许开始

Phase 2 是一张三列表，按档位给出不同要求：

输入	full	standard	minimal
需求注册表	缺失则停止	可选	不期待
故事点名的 ADR	缺失或非 Accepted 则停止	只在点名时检查	只在点名时检查
控制清单	缺失则警告，继续	缺失则警告，继续	不期待

依赖检查独立于档位。每个依赖故事都要跑一次 `story-facts.sh`，状态不是 Complete 就停下来问用户：继续承担风险、先去完成依赖、还是它其实完成了只是状态没更新。

这里有一个细节：脚本读的是状态字段，不是 git 历史或文件内容。状态没更新，它就会认为没完成。仓库选择相信声明，同时把判断权交给用户。

第四步：确定代码根

写代码之前必须先回答"代码写到哪里"。`code-root-resolution.md` 给出的顺序是：

1. `engine.name` 从 `project.local.yaml` 再 `project.yaml` 读。
2. 旧版镜像 `.claude/docs/technical-preferences.md` 里的 `Engine:` 行。
3. 目录推断，但只在无歧义时：`src/`、`Assets/`、`Source/` 中恰好存在一个。
4. 否则：未解析。

未解析时不是默认到 `src/`，而是停止写入并报告。文档给出的理由很直接：Unity 项目里写到 `Assets/` 之外的代码不会被编译，也不会出现在编辑器里，工作安静地失效。

测试根同理。Unity 的测试要放 `Assets/Tests/EditMode/` 或 `PlayMode/`，放错位置的结果是测试运行器显示"0 tests, Passed"。这个输出看起来像通过，实际上是没跑。

第五步：选择执行者

Phase 4 用两张表。第一张按故事选程序员：

故事	角色
Foundation 层，任意类型	`engine-programmer`
类型 UI	`ui-programmer`
类型 Visual/Feel	`gameplay-programmer`
Core 或 Feature，玩法	`gameplay-programmer`
Core 或 Feature，AI 或寻路	`ai-programmer`
Core 或 Feature，网络	`network-programmer`

第二张按配置选引擎专家。`specialists` 行给出 `code`、`shader`、`ui`、`primary` 四个槽位，来自 `project.yaml`。例如 Godot C# 项目会配 `godot-csharp-specialist` 作为 `code`，这样即使 `engine.name` 只写 `Godot`，也能区分 GDScript 和 C#。

技能明确要求：不要凭记忆指定专家，以配置行为准。所有槽位都是 `unset` 且没有 `primary` 时，说明"未配置引擎专家"，然后继续，不阻塞。

第六步：构造简报

简报是整个链路里最像"工程"的部分。它不粘贴文档，只给路径和要点：

1. 故事文件本身，含验收标准、边界、QA 用例。
2. 每个需求 ID 在注册表里的匹配行，原文是权威。
3. ADR 的决策摘要和实施说明，**内联**，不给 ADR 路径。
4. 控制清单里对应层的规则，按章节摘。
5. `project.yaml` 的命名规范和注入的 `perf_budgets` 行。
6. 故事里测试证据的路径。

然后是测试要求，按 `qa.level` 分三档：

- minimal：明确告诉代理"不要写测试文件，测试证据在此档豁免"。这句话必须说出来，因为"省略一行"和"忘了要求"对代理来说无法区分。
- standard：测试与代码同时写，每条验收标准至少一个测试函数。禁止随机种子、时间依赖断言、外部 IO。
- full：同上，且每条标准都必须覆盖，`/story-done` 遇到未覆盖会阻断。

最后是输出契约：改了哪些文件、写了哪些测试函数、有没有问题或阻塞、是否完成。边界也写明：只在解析出的代码根下写代码，只在测试根下写测试；代码根未解析就什么都不写；不做一次性抽象；不为不可能发生的情况写错误处理；不为了让测试通过而特判。

第七步：检查产物而不是读报告

Phase 6 的第一条规则是：先看文件在不在。

> 代理报告里点名的文件如果在磁盘上不存在，这一步就是失败的，不管报告写得多流畅。

这条规则来自 `error-recovery-protocol.md` 的"触发器 0"。文档描述了一种具体的失败形态：并行的子代理做了大量工作，然后返回一段像样的开场白，没有路径、没有摘要、没有 BLOCKED 标记，也没有写任何文件。父会话读到这段文字，以为阶段完成了，实际上半个并行阶段消失了。

所以判断依据是产物，不是转录。

第八步：解析检查与运行观察

解析检查因引擎而异，Godot 的命令写得很具体：

```
godot --headless --path . --import
godot --headless --path . -s res://.claude/scripts/godot-parse-check.gd -- res://<changed.gd>
```

第二条退出码 1 表示有脚本没加载，输出里的 `PARSE FAIL:` 会点名是哪个。技能特别提醒：不要用 `--check-only`，只跑 `--import` 也不算解析检查。

引擎二进制不存在时，结果必须写成 `parse NOT VERIFIED — engine binary not available`，不允许读一遍代码就断言"应该能解析"。

运行观察针对玩家能看到的故事。步骤是：直接启动到故事改动的场景，截图保存到 `production/qa/evidence/<story-slug>/`，用 `review-receipts.sh hash` 给改动文件打内容哈希，存成 `stamp.txt`，然后打开图片与验收标准比对。

最后只输出一行：

```
Run result: OBSERVED — <屏幕上是什么>
Run result: NOT VERIFIED — <原因>
Run result: N/A — <为什么没有可观察内容>
```

`N/A` 的原因不能是"这是个逻辑故事"，因为逻辑故事也可能有可见结果。这个限制防止用类型标签逃避观察。

第九步：收尾

Phase 7 输出固定结构：改动文件、测试证据、验证命令与结果、逐条验收标准、偏差、引擎风险、阻塞。然后把实际改动写入故事的 `## Files` 段，替换计划清单。

再写会话检查点。检查点包含更新日期、分支、当前任务、下一步、阻塞项、进行中的文件、运行结果、未决问题。它是下次会话恢复工作的唯一依据。

最后一行是收尾块，三行，状态词只有 Implemented、INCOMPLETE、BLOCKED 三种。

常见误解

误解一：代理说完成就是完成。检查点在产物：文件、测试、解析结果、截图。

误解二：解析通过等于能运行。解析只说明代码能加载，不说明行为正确，所以还要运行观察。

误解三：测试文件存在等于测试通过。`/dev-story` 和 `/story-done` 都只检查存在性，通过与否要用户自己跑测试套件，仓库在提示里明确要求用户先跑再关闭故事。

练习

1. 在模板仓库上运行 `story-facts.sh --all`，解释每列含义。

2. 假设引擎是 Unity，代码根解析失败，`/dev-story` 应该做什么？为什么不能默认写到 `src/`？
3. 为一个 Visual/Feel 故事列出从实施到关闭需要的全部证据。

答案提示

1. 第一列是统计项，第二列是数量，第三列是搜索路径。全零表示 `production/epics/` 下没有故事。
2. 停止写入并报告未解析的代码根。默认到 `src/` 在 Unity 里不会编译，工作会安静失效。
3. 代码与测试、解析检查结果、运行到目标场景的截图、截图目录下的 `stamp.txt`、`<story-slug>-evidence.md` 里的签字行，以及 `/story-done` 的逐条核对表。

09

验收：/story-done 如何区分事实与断言

场景

故事报告说"实现完成，所有验收条件满足"。`/story-done` 不采信这句话。它把每条验收标准摊开，逐条问：这条有测试文件吗？这条有截图吗？这条你亲眼看过了吗？这条需要试玩吗？

答案分成几种：有人看到过、有文件存在、用户口头确认、没人验证。四种状态对应不同判决。

这一章讲验收协议。它比实施流程更值得借鉴，因为大多数团队的"完成"定义比这里模糊。

证据分级

`/story-done` 的 Phase 1 有一张按 `qa.level` 分列的表：

检查	minimal	standard	full
Logic、Integration 测试	豁免	必需	必需
Config/Data 冒烟报告	豁免	必需，默认建议	必需，默认阻断

检查	minimal	standard	full
Visual/Feel、UI 截图	必需	必需	必需
未测试的验收条件	从不升级	超过一半阻断	任何一条阻断

注意第三行：截图在所有档位都必需，包括 minimal。理由在 README 里："解析检查不是运行"。一个菜单画到屏幕外，测试仍然全绿。

第四行是档位差异最明显的地方。minimal 允许未测试条件存在但不升级；standard 在一半以上未测试时阻断；full 遇到任何一条未测试就阻断。

三种核对方式

Phase 3 规定每条验收标准只能用三种方式之一核对。

第一，看。文件存在、测试文件存在、代码里没有本应在数据文件里的数值、玩家可见字符串没有硬编码。看只能证明"存在"，不能证明"正确"。所以报告里写"test file present"，永远不写"test passes"。

第二，问。行为、手感、性能这类条件必须问用户。一次最多四个问题，选项是"是，通过"、"否，失败"、"还没测"。"还没测"等于 UNTESTED，判决最高只能到 NOT ASSESSED。

第三，看屏幕。涉及可见结果的条件，需要 `production/qa/evidence/<story-slug>/` 下有一张**打开看过**的截图。技能明确禁止描述没打开过的图片，也禁止把检查点里 `Run result: OBSERVED` 的文字当成自己看过。

这三种方式划出了一条清晰的界线：模型可以检查产物，不能替用户确认体验。

可追溯表

核对完，生成一张表：

```
| 验收条件 | 由什么覆盖 | 状态 |
| AC-1: ... | tests/unit/combat/parry_test.gd::test_parry_window |
COVERED |
| AC-2: ... | production/qa/evidence/story-003/01-sparks.png |
COVERED |
| AC-3: ... | — | UNTESTED |
```

这张表是整个验收协议的产物。它让"哪条没做"变得具体，而不是一句"基本完成"。

证据会过期

有一段设计很细致：截图可能比代码旧。你先截了图，后来又改了数值，截图没重拍，验收时看起来通过。

`review-receipts.sh` 解决这个问题。`/dev-story` 截完图后，对改动的每个文件打内容哈希，写进 `stamp.txt`：

```
Reviewed-Content-Hash: <path> <hash>
```

`/story-done` 用 `check` 比对：

```
RECEIPT: <stamp.txt>
UNCHANGED: <path>
```

如果代码变了：

```
CHANGED: <path>
```

文件被删了则报 `GONE:`，收到从未展开的通配符报 `UNRESOLVED:`。没有回执时报 `RECEIPT: NONE`，并说明"无法判断证据是否比代码旧"。

在固定快照上实测一遍：

```
bash .claude/scripts/review-receipts.sh hash combat.gd > stamp.txt
bash .claude/scripts/review-receipts.sh check stamp.txt
# RECEIPT: stamp.txt
# UNCHANGED: combat.gd

echo "b" >> combat.gd
bash .claude/scripts/review-receipts.sh check stamp.txt
# RECEIPT: stamp.txt
# CHANGED: combat.gd
```

脚本头部解释了为什么用内容哈希而不是 git 或 mtime。`git diff` 需要一个已提交的基线；`git status` 只看未提交改动；mtime 在备份恢复和云同步下会说谎，一次旧 mtime 的恢复会让新内容悄悄逃过评审，这是

危险的假阴性。哈希没有这些问题。剩下的失败方向是假阳性：重评审没变的东西，浪费 token，但不会漏评审。

还有一个刻意的例外：未被回执记录过的字面路径缺失时保持沉默。因为某些调用方每次都会传一个可选输入，报出来会打断合法的跳过。但回执记录过、现在消失的路径会报 `GONE`，因为上次评审确实读过它。

范围检查

Phase 5 对照故事的 `## Files` 和 `git status --porcelain`。改动文件不在清单里就标 OUT OF SCOPE，只作提示，不阻断。没有 `## Files` 段时，回退到会话检查点里的"进行中的文件"；两者都没有，报告 `Scope: NOT ASSESSED`。

工作区干净时报告 `Scope: not checked — no uncommitted changes to compare`。这句话避免把"没有改动"误读成"范围正确"。

四态判决

Phase 7 规定的判决顺序是固定的，第一条匹配就生效：

1. BLOCKED：某条验收条件失败，或存在阻断级的缺口或偏差。
2. NOT ASSESSED：某条条件无法评估。比如它没说清可观察结果、用户答"还没测"、评审门返回 NOT ASSESSED 且输入没补上、证据存在但读不了、缺少判定偏差所依据的 GDD 或 ADR。
3. COMPLETE WITH NOTES：所有条件通过或延期，但存在建议级事项。

4. COMPLETE：全部通过，没有建议级事项。

顺序里有两条设计意图。

第一，NOT ASSESSED 排在两个通过态之上。无法评估的一部分，不能当作整体良好。这对应 `skill-authoring.md` 的第一条义务：每个判决都要有"无法评估"这个值，并且它要排在通过之上、失败之下。

第二，信息行不算建议项。证据时间戳、缺少冲刺文件、运行结果这些只是信息，不会把 COMPLETE 变成 COMPLETE WITH NOTES。这个区分防止一份干净的故事因为环境差异被降级。

关闭是一次显式决定

判决出来之后，技能会问："你想怎么处理？"

选项是：关闭并记录、关闭并把建议项记为技术债、先别关我要修、或者接受偏差直接关。

只有在用户明确选择关闭时才写文件。BLOCKED 或 NOT ASSESSED 时，推荐选项是"先别关"，并列出要改什么。技能还规定：不许自己修复失败的条件，只报告并询问。

如果 `automation_always_ask` 列表包含 `scope_changes`，那么即使当前是 autonomous 模式，关闭一个带偏差的故事仍然会提问。因为关闭动作改变了范围承诺。

常见的验收漏洞

第一个漏洞：把测试文件存在当成测试通过。整套流程只检查存在性，运行测试是用户的责任。技能在实施结束时明确提示："先跑你的测试套件并确认新测试通过，再关闭故事。"

第二个漏洞：把模型描述当成观察。截图必须被打开看过，`OBSERVED` 文字只是上一阶段的描述，引用时要标明来源。

第三个漏洞：证据比代码旧。哈希回执解决这个，但前提是实施阶段真的写了 `stamp.txt`。没有回执时，脚本诚实地说"无法判断"，而不是默认通过。

常见误解

误解一：minimal 档什么证据都不用。截图仍然必需，只是自动化测试豁免。

误解二：判决由模型自由裁量。四态有严格优先级，还有一张按档位的阻断表。

误解三：关闭故事会自动提交代码。技能只打印一条建议的 `git add` 和 `git commit` 命令，不执行。

练习

1. 在临时目录里创建文件、打哈希回执、修改文件，观察 `check` 的输出变化；再删掉文件，看 `GONE`。

2. 一条验收条件写"格挡手感要扎实"，按验收协议它应该怎么被处理？判决会是什么？
3. 解释为什么 NOT ASSESSED 排在 COMPLETE 之上、BLOCKED 之下。

答案提示

1. 依次得到 UNCHANGED、CHANGED、GONE。没有回执时得到 `RECEIPT: NONE` 和每个文件的 NEW。
2. "手感"无法靠看文件确认，也不能靠单张截图证明，属于需要试玩的条件，标记 DEFERRED 并要求试玩记录。若没有任何可观察定义，判决停在 NOT ASSESSED。
3. 无法评估意味着没有建立"良好"的证据，所以不能算通过；但已知的失败更可操作，所以排在 BLOCKED 之后。这个排序避免了把访问问题伪装成通过，也避免把真实失败埋在未知后面。

10

Hook：十五个检查点里谁真的能拦住

场景

你在故事文件上执行一次 Edit，把 `Status: Ready` 改成 `In Progress`。写入发生前，`validate-write.sh` 收到一条 JSON，里面有工具名、文件路径和新内容。它检查这个文件是否属于"永不覆盖"清单、配置是否写到该写的位置、原型目录有没有被生产代码引用。发现问题时它打印一段警告，然后退出码为 0。

同一时刻，你试图整篇读取一份 60 KB 的设计文档。`validate-read.sh` 拒绝，退出码 2，把文件的所有标题和行号打给你。

两个 Hook，一个劝阻，一个拦住。这一章把 15 个注册命令逐个归类。

注册表

`.claude/settings.json` 的 `hooks` 段把命令挂到事件上。快照里共有 15 条命令，分布在 8 个事件：

事件	命令	数量
SessionStart	`session-start.sh`、`detect-gaps.sh`	2

事件	命令	数量
PreToolUse（Bash/PowerShell）	`validate-commit.sh`、`validate-push.sh`	2
PreToolUse（Write/Edit）	`validate-registry.sh`、`validate-write.sh`	2
PreToolUse（Read）	`validate-read.sh`	1
PostToolUse（Write/Edit）	`validate-assets.sh`、`validate-skill-change.sh`	2
Notification	`notify.sh`	1
PreCompact	`pre-compact.sh`	1
Stop	`session-stop.sh`	1
SubagentStart	`log-agent.sh`	1
SubagentStop	`log-agent-stop.sh`、`validate-agent-artifact.sh`	2

`hooks/` 目录里另外有三个脚本没有注册：`yaml-helper.sh` 是被 source 的库，`log-instructions.sh` 和 `update-check.sh` 需要手动或由其他脚本调用。所以"15 个 Hook"和"18 个脚本"都对，口径不同。

能拦住的

真正能阻断的有三处。

`validate-read.sh` 用 `exit 2` 拒绝超限的整篇读取。它是唯一一个在 PreToolUse 阶段硬拦的验证 Hook。拒绝时它不是简单说"不行"，而是列出标题和行号，告诉模型该怎么分段读。这符合"拒绝要给替代路径"的原则。

`validate-assets.sh` 在 PostToolUse 阶段对无效 JSON 用 `exit 2`。注意 PostToolUse 发生在写入之后，文件已经落盘，所以它不是"阻止写入"，而是"让模型看到自己写坏了"。脚本头部把这件事说得很清楚：

> PostToolUse Hook 无法阻断，写入已经发生。唯一的问题是通知谁，而退出码决定这个：exit 1 只给用户看，模型以为写干净了继续往下走；exit 2 把 stderr 作为可操作反馈交给模型，它能修正刚写的文件。

`validate-commit.sh` 的阻断范围很窄：只有暂存了 `assets/data/`、`Assets/Data/`、`Content/Data/` 等路径下的 JSON，且校验失败或无法校验时才 `exit 2`。其余检查，包括 GDD 章节完整性、硬编码数值、无主 TODO，都只是追加到警告列表，最后 `exit 0`。

只提醒的

`validate-write.sh` 是最容易被高估的一个。它的名字像守门员，实际行为写在文件头：

```
exit 0 always = advisory (it warns, never blocks)
```

它检查的项目不少：故事文件是否被覆盖、配置枚举是否合法、本地配置是否写了不可本地覆盖的键、评审模式是否被钉死、原型目录与生产代码是否互相引用、字符串冻结期间是否改了文案、子代理是否越界写文件。

每一项都有真实价值，但它们全部走警告通道。`skill-authoring.md` 要求"跳过的步骤要自己说出来"，这个脚本遵守了：无法检查时会打印 `Write check skipped: ...`，而不是安静放行。

`validate-registry.sh`、`validate-agent-artifact.sh`、`validate-skill-change.sh` 同样是建议级。`validate-push.sh` 更直接：它检测到你在往受保护分支推送时，输出一段 JSON，其中 `additionalContext` 会作为上下文给模型看，`systemMessage` 给用户看。脚本里甚至留了一行注释，说明如果要改成阻断，把 `printf` 换成 `echo` 加 `exit 2` 即可。

记录型的

SessionStart 的两个脚本负责恢复和体检。`session-start.sh` 读 `production/session-state/active.md`，把检查点区域预览出来；这个区域被要求控制在 25 行以内，可以原样展示。`detect-gaps.sh` 检查项目缺什么。

`pre-compact.sh` 在上下文压缩前触发，提醒保存状态。官方文档里 PreCompact 的退出码 2 可以阻止压缩，但这个脚本没有使用阻断，只是提示。

`log-agent.sh`、`log-agent-stop.sh`、`session-stop.sh` 写审计日志。`notify.sh` 在权限提示或空闲提示时发桌面通知，Windows 走 PowerShell，其他平台目前是空操作。

Hook 的输入从哪来

每个 Hook 从标准输入拿到一段 JSON。`validate-write.sh` 的注释给出了两种形态：

```
{"tool_name":"Write","tool_input":
{"file_path":"...","content":"..."}}
{"tool_name":"Edit","tool_input":
{"file_path":"...","old_string":"...","new_string":"...","replace_all
":false}}
```

编辑操作没有完整的新文件内容，只有替换片段。所以 Hook 调用 `hook-input.sh --new <tmp>` 重建写入后的文件，再交给 `write-guard.sh` 判断。重建失败时，依赖内容的规则（配置、原型）会明确说没有检查，依赖路径的规则继续运行。

这段设计体现了一个原则：检查器要能区分"检查通过"和"没检查"。`write-guard.sh` 输出的第一行是 `checked <n> rules`，让调用方知道跑了多少条。

警告怎么送达

早期的做法是往 stderr 打印，然后 `exit 0`。问题在于，退出码 0 的 stderr 只进调试日志，用户和模型都看不到。脚本注释里记录了这个问题：警告被写进了没人读的地方。

现在的做法是调用 `yaml-helper.sh` 里的 `hook_warn`，输出一段结构化 JSON，包含 `systemMessage` 和 `hookSpecificOutput.additionalContext`。前者给用户，后者给模型。没有这个函数时回退到 stderr。

这个细节值得学：Hook 的输出通道比输出内容更容易出错。一条正确的警告发到错误的通道，等于没发。

官方语义对照

用 Context7 查 Claude Code 官方 Hook 文档，确认了几条与本仓库实现对应的规则：

- PreToolUse 的 `exit 2` 阻断工具调用，stderr 作为阻断原因反馈给模型。
- PostToolUse 的 `exit 2` 不能撤销已发生的操作，只把 stderr 作为反馈交给模型。
- 其他非零退出码属于非阻断错误，通知里带 stderr 的第一行。
- `exit 0` 且输出合法 JSON 时，JSON 字段生效。

这解释了为什么 `validate-assets.sh` 明知写入已经发生还要用 `exit 2`：它要的不是撤销，而是让模型修。

同时要提醒：Hook 的语义在不同版本间有过调整，尤其是"谁看到 stderr"。仓库里 `validate-assets.sh` 的注释认为 `exit 1` 只给用户看，这和某些版本文档的描述不完全一致。移植这套 Hook 时，应以你当前 Claude Code 版本的官方文档为准，并写一个最小实验确认。

权限与 Hook 的分工

`settings.json` 的 `permissions.deny` 是另一层。它列出 `rm -rf` 的各种变体、`git push --force` 的各种变体、`sudo`、`chmod 777`、读 `.env`、编辑 `.uasset` 等。

这一层由宿主执行，比 Hook 更硬，但它只匹配命令字符串。`validate-push.sh` 的存在说明作者不完全信任字符串匹配：它额外解析 git 命令的分段、处理 `&&` 链接、跳过引号里的文本、识别 `git -C` 等变体，才判断这是不是一次推送。

两层的分工可以概括为：权限层挡住明确危险的动作，Hook 层处理需要上下文判断的规则。

常见误解

误解一：有 Hook 就有安全边界。15 个里只有 3 处会阻断，其余是提醒或记录。

误解二：PostToolUse 能撤销坏写入。它只能在写入后告知，修复要靠模型或人。

误解三：Hook 一定在项目根运行。脚本开头都有一段根目录解析逻辑，因为工作目录不对会读写错误的树。这段逻辑的优先级是：当前目录有 `project.yaml` 或 `.claude/` 就用当前目录，否则用 `CLAUDE_PROJECT_DIR`，最后回退到脚本自身位置。

练习

1. 从 `settings.json` 数出所有 Hook 命令，按"阻断、反馈、记录"分类。
2. 解释 `validate-read.sh` 为什么要在 stderr 里列出标题和行号，而不是只说文件太大。
3. 把 `validate-push.sh` 改成真正阻断受保护分支推送，需要改哪一行？会带来什么后果？

答案提示

1. 阻断：`validate-read.sh`；反馈：`validate-assets.sh`、`validate-commit.sh`（仅数据 JSON）；其余为提醒或记录。
2. 拒绝必须给出可执行的替代路径，否则模型会重试同样的读取。列出标题让它能按范围读，一次拒绝就完成教学。
3. 把输出 JSON 的那段替换为 `echo "BLOCKED: ..." >&2; exit 2`。后果是所有受保护分支的推送都被硬拦，包括紧急修复；需要额外的绕过机制或人工在本地改脚本。

11

注册表：跨文档事实如何不漂移

场景

三份设计文档都提到哥布林。战斗文档说它有 12 点生命，经济文档说掉落 3 枚金币，任务文档说它是"精英怪"。三周后战斗文档把生命改成 18，另外两份没动。等到实现时，程序员按哪份写？

这类问题靠人工同步必然漂移。CCGS 的答案是把跨文档事实集中到注册表，其他文档引用而不是复制。

三个注册表

仓库里有三份：

`design/registry/entities.yaml` 存游戏世界事实：实体、物品、公式、常量。文件头的规则很具体：

- 只登记跨系统边界的事实。单份 GDD 内部使用的公式不必登记。
- 永不删除条目，用 `status: deprecated` 标记。
- 值改变时更新值、写修改日期，并注释旧值和哪份 GDD 改的。
- `source` 是权威 GDD，其他引用它的文档列在 `referenced_by` 。

- 新文档引用已有条目时追加路径，不新建重复条目。

`docs/architecture/tr-registry.yaml` 存需求条目，编号 `TR-<system>-NNN`。故事引用 ID，评审时回表读最新文本。

`docs/registry/architecture.yaml` 存架构层面的状态、契约、系统、目标帧率、用途、模式。它是控制清单和架构评审的共同底稿。

一个脚本，三种读法

`.claude/scripts/registry.sh` 把三份注册表统一成一种读法。子命令有九个：

- `list`：每个条目一行，可加 `--system` 只留某个系统的需求。
- `get`：按名字取条目及其原始行。
- `refs`：给名字返回它引用的路径，给需求 ID 返回引用它的故事或史诗，给路径返回所有引用该路径的条目。
- `slice`：某个文件作为 source 或 gdd 或 adr 涉及的全部条目。
- `validate`：重复名、故事引用了不存在的需求、路径不存在、状态越界、布局不支持。
- `compare`：比较变更前后的注册表，列出被删除的条目和新增的非法状态。
- `next-id`：给出下一号。

在快照上跑 `validate`：

```
registry   entities   design/registry/entities.yaml     0
registry   tr         docs/architecture/tr-registry.yaml           0
registry   architecture          docs/registry/architecture.yaml   0
checked    production/epics      0
duplicate  0
tr_unknown 0
path_missing          0
status     0
layout     0
```

三份注册表都能读，条目数为 0，五个检查项都是 0。输出是 TSV，每行第一列是类别。脚本注释写明：它只输出观察，不输出判决。缺一个路径重不重要、故事能不能引用已废弃的需求，由调用技能决定。

`next-id` 的行为也实测过：

```
$ bash .claude/scripts/registry.sh next-id story stealth
next       story      stealth    story-001  -          0
```

格式是 `next <kind> <scope> <id> <highest> <count>`。最高号是 `-`，已见 ID 数为 0，所以下一号是 `story-001`。

为什么 ID 只增不减

`next-id` 的规则是"见过的最大号加一"，不是"找最小空号"。脚本注释说明：删除或废弃过的 ID 永远不再发放。

这条规则看起来浪费编号，实际是审计需要。旧故事、旧评审、旧提交信息里都引用过 `TR-combat-007`。如果这个号被回收给另一条需求，历史记录会指向错误的对象，而且无法察觉。

`compare` 子命令也基于同样的假设：注册表是只追加的。它列出"旧表有、新表没有"的条目，因为正常变更不该删除条目。

布局为什么这么严格

注册表是 YAML，但读取方式不是通用 YAML 解析器，而是一套固定布局规则：

- 一个 section 是第 0 列的一个键，持有块列表；空时写 `[]`。
- 一个条目以 `- name: <name>` 开头，短横线在第 0 列或第 2 列，同一 section 内必须一致。
- 字段比短横线深两列，更深的行属于上一个字段。
- 含 `": "` 的值必须加引号。
- `|` 或 `>` 开头的是多行文本，比键更深的行都属于它。

任何不符的行都报 `LAYOUT`，附行号和期望的写法，并且该注册表的条目全部不读，而不是猜。

为什么要放弃通用解析器？脚本头部给了一个理由：`bash 3.2` 和 `BWK awk` 兼容，无 Python 依赖。macOS 自带 bash 3.2，这个限制是真实的。用固定布局换取零依赖，代价是写错格式时条目会整份失效。

失效被设计成显式的。 `LAYOUT` 行会明确告诉你哪一行不符合，而不是安静地少读几条。这就是 `skill-authoring.md` 第二条义务的实践：缺失值不能默认成宽松解释。

兼容旧格式

脚本里有一个刻意的例外：架构注册表里早期版本写过 `status: superseded_by: ADR-NNNN` ，没有给值加引号。

严格按布局规则，这一行会被判为不支持。脚本选择读它，把状态读成 `superseded_by: ADR-NNNN` ，同时输出一行 `UNQUOTED` ，告诉你在哪一行、应该写成什么样。

这个处理方式值得借鉴：对历史数据宽容读取，但把不规范之处明确报出来，让用户有机会修。既不因为一个旧格式让整份注册表失效，也不假装它完全合规。

读取者清单

`entities.yaml` 的文件头列了谁会读它，以及每条读法的 Grep 模式：

```
WRITTEN BY: /design-system, /consistency-check, economy-designer,
systems-designer
READ BY:    /design-system, /consistency-check, /review-all-gdds,
```

```
/design-review,
            /content-audit, /balance-check, /propagate-design-change,
/asset-spec, ...
```

这份清单的价值在于变更影响分析。要改注册表格式，先看谁在读；要新增一个读取方，先确认它的读法与现有布局兼容。

`/propagate-design-change` 的读取方式特别有意思：它用 `referenced_by` 找出所有引用某个事实的文档，然后逐个评估影响。这是把"改一处要同步多处"从人的记忆变成了图遍历。

常见误解

误解一：注册表是给人看的文档。它主要是给脚本和技能读的，人只是维护者。

误解二：可以随时清理无用条目。只能标废弃，因为历史引用仍然有效。

误解三：格式错了脚本会猜。它会报 `LAYOUT` 并跳过整份注册表，避免基于半个文件做判断。

练习

1. 在副本里给 `entities.yaml` 加一个条目，字段缩进故意写错两格，运行 `registry.sh validate`，观察输出。
2. 用 `registry.sh refs` 查询某个 GDD 路径，说明输出中 `how` 列可能出现哪些值。

3. 解释为什么 `next-id` 不回收已删除的编号，并用一个具体场景说明回收的后果。

答案提示

1. 会得到 `LAYOUT` 行，带文件、行号、发现的和期望的布局，该注册表条目数为 0。
2. `source`、`gdd`、`adr`、`referenced_by`，以及 `cited`（故事或史诗引用某个需求）。
3. 假设 `TR-combat-007` 被删除后回收给了 `TR-ui-007`，旧故事仍写着 `TR-combat-007`，评审回表会读到完全不同的需求，而且没有任何报错。只增不减可以避免这种静默错配。

12

协作：团队流水线与它的塌缩

场景

你输入 `/team-combat melee parry system`。技能正文列了七个角色：游戏设计师、玩法程序员、AI 程序员、技术美术、音效设计师、引擎专家、QA。看起来要派七个代理。

实际运行可能只派一个。

原因写在 `team-combat` 的 Phase 0：`team.size` 决定活跃角色集。默认值 `individual`，玩法程序员跑完整条流水线，其他角色通过它咨询，不单独派生。`small` 才启用完整流水线。`studio` 再加引擎子专家和一轮对抗性评审。

为什么默认塌缩

`team.size` 是 rigor 展开的一部分。minimal 和 standard 都映射到 `individual`，只有 full 映射到 `studio`。

理由很直接：token。七个代理各读一遍设计文档、各写一份报告，成本是单代理的数倍，收益不一定成比例。作者在 README 的对照实验里已经表明，更多流程买到的是可追溯性，不是更好的游戏。

但塌缩有一个副作用：读技能正文的人会以为七个角色都跑了。所以正文强制要求在执行前声明：

> Active set (team.size: `resolved`)：本次真的会派生的角色。Not spawned this run: 正文点名但不会派生的角色，通过最近的核心角色咨询。提高 team.size（或 modes.rigor）可以扩大。

这段声明是这套系统里最值得抄的设计之一。它把"因为配置而收窄"和"因为故障而漏派"区分开。没有这句话，用户看到一份只有玩法程序员署名的战斗设计，无法判断是正常塌缩还是流水线坏了。

决策点

团队流水线的骨架是阶段加决策点。每个阶段结束，技能把子代理的提议整理成可选选项，写入对话，然后用提问工具请用户决定。

`collaborative` 模式下，每个阶段都要用户批准才继续。`guided` 模式下，除非某阶段 BLOCKED，否则自动前进。`autonomous` 模式下全程跑完，每个决策用 `log_decision` 记录。

三种模式共享一条规则：`automation_always_ask` 里的类别永远提问。默认是范围变更、文件删除、schema 变更。即使 autonomous，砍功能、删故事、改故事模板都会停下来问。

简报与返回契约

派生代理时，技能要求两件事。

第一，简报要蒸馏，不要倾倒上下文。共享输入只读一次，把每个代理真正需要的几行内联传过去。已经读过的文档不要传路径，因为拿到路径的代理会重读整份文件。

第二，每个代理的提示词结尾必须有返回契约：

> 把完整输出写到 `[路径]`。这个点名路径就是你的写入授权（在下面的有界例外下），不需要再单独请求批准。只返回三样东西：写入的路径、不超过五条决策摘要、以及 BLOCKED 或 CONCERNS 项各一行。不要复述你读过的文档。

没有这条契约，代理会把读过的所有内容复制回父会话，上下文瞬间膨胀。有了它，父会话只收到指针和摘要。

实现类代理是例外：它们要先返回将创建或修改的文件清单，等父会话确认后再写。因为代码改动的影响面比一份报告大。

并行协议

`coordination-rules.md` 规定：

1. 所有独立的 `Agent` 调用要在等待任何结果之前全部发出。

2. 收集完所有结果，再进入依赖阶段。
3. 任何代理 BLOCKED 要立刻上报，不能悄悄跳过。
4. 部分代理完成、部分阻塞时，仍然产出部分报告。

举的例子是 `/review-all-gdds`：第二阶段一致性检查和第三阶段设计理论互不依赖，可以同时跑；第一阶段加载 GDD 是两者的输入，不能并行。

这套协议的关键不是并行本身，而是"独立"的判定。两个代理如果读同一批文件但各自分析，可以并行；如果一个的输出是另一个的输入，串行。

错误恢复

`error-recovery-protocol.md` 的四个触发器值得完整记住。

触发器 0：在读取响应之前先验证产物。返回契约点名了路径，就去检查路径是否存在。一个流畅的回复不是阶段跑过的证据，产物才是。

触发器 1：代理返回 BLOCKED、报错或说无法完成时，立刻上报，不做依赖它的后续阶段。

触发器 2：评估依赖。被阻塞代理的输出是不是后续阶段的必需输入，是就不要越过。

触发器 3：给用户选项。跳过并记录缺口、缩小范围重试、停下来解决阻塞。

触发器 4：永远产出部分报告。不要因为一个代理阻塞就丢弃已完成的工作。

还有一条容易被忽略：跳过不等于解除阻塞。在团队流水线里选择"跳过这个代理"，剩余阶段可以继续，但最终判决必须是 BLOCKED，点名被跳过的代理和缺口，不能写成 COMPLETE。

叙事团队的例外

`team-narrative` 有一条特殊规则：无论当前 review 模式是什么，只要叙事总监在团队里，ND-CONSISTENCY 一致性检查就必须跑。

理由写在 `director-gates.md`：这是该流水线唯一的叙事一致性检查，不是可选评审。如果它也按 lean 或 solo 跳过，叙事团队就没有任何一致性保障。

这个例外说明模式化规则需要留一个"结构上不可省"的出口。不是所有检查都能用同一个开关控制。

团队流水线清单

仓库提供九个团队技能：战斗、叙事、UI、发布、打磨、音频、关卡、Live Ops、QA。它们的结构相似：角色表、阶段、决策点、错误恢复。

差异在角色数量和阶段深度。发布团队点名十个角色，默认只跑一个；叙事团队点名六个，默认只跑编剧。这种"文档描述十人、实际运行一人"的落差是设计意图，不是缺陷。

常见误解

误解一：团队技能会派出一整支代理队伍。默认只派一个，除非显式提高 `team.size`。

误解二：并行越多越快。并行的前提是输入独立，否则会拿到基于旧输入的结论。

误解三：跳过阻塞的代理后还能判 COMPLETE。规则明确要求最终判决为 BLOCKED。

练习

1. 阅读 `team-combat` 的 Phase 0，写出 `individual`、`small`、`studio` 三档分别会派生哪些角色。
2. 为一次两阶段的团队任务设计并行方案，说明哪个阶段可以并行、哪个不能。
3. 解释返回契约里的"点名路径即写入授权"为什么限定在 `production/`、`docs/`、`tests/` 下的新文件。

答案提示

1. individual 只派生玩法程序员（必要时升级到 AI 程序员）；small 派生设计、玩法、AI、技术美术、音效、引擎专家、QA；studio 再加引擎子专家和对抗性评审。
2. 共享输入加载阶段必须串行，两个分析阶段如果都只依赖加载结果且互不依赖，可以并行。收集完再进入综合阶段。

3. 这三个目录放的是项目产物，不是可执行源码或配置。限制在这里可以避免子代理在"阶段已批准"的名义下改动生产代码或安全配置。

13

引擎适配：代码根不是一个常量

场景

同一条 `/dev-story` 命令，在三个项目里要写三个不同的目录：

- Godot 项目，代码写 `src/`。
- Unity 项目，代码写 `Assets/`。
- Unreal 项目，代码写 `Source/<Module>/`。

写错了不会报错。Unity 只编译 `Assets/` 和 `Packages/`，写到 `src/` 的代码在编辑器里根本不存在。Unreal 只从 `Source/` 发现模块，别处的文件不参与构建。Godot 没有这个限制，因为 `res://` 覆盖整个项目根。

这一章讲仓库如何把"引擎"从一个提示词变量变成一组可解析的路径和规则。

解析顺序

`code-root-resolution.md` 定义了四步：

1. 从 `project.local.yaml`、`project.yaml` 读 `engine.name`。

2. 读旧版镜像 `.claude/docs/technical-preferences.md` 的 `Engine:` 行，`[TO BE CONFIGURED]` 不算值。
3. 从目录推断，但只在无歧义时：`src/`、`Assets/`、`Source/` 恰好存在一个。
4. 否则未解析。

第三步有两个陷阱。第一，Windows 和默认 macOS 的文件系统不区分大小写，测试 `Assets/` 是否存在时，Godot 项目里的小写 `assets/` 也会命中。所以比较必须按目录列表里的实际拼写。第二，同时存在两个候选根时不要猜，猜错会扫一半代码然后报告干净。

第四步是最重要的：未解析时**不默认到** `src/`。文档把这条写成整个文件存在的理由，并引用了 `skill-authoring.md` 的第二条义务：缺失值不能默认成宽松的那个。

对读取和扫描，未解析时报告 `NOT ASSESSED — code root unresolved`，说明哪个检查没跑。对写入，直接不写。

为什么一条 grep 会骗人

假设 `validate-commit.sh` 里有一行 `grep -E '^src/gameplay/'` 用来找硬编码数值。在 Godot 项目里它正常工作。在 Unity 项目里它什么都匹配不到。

匹配不到的输出是什么？没有警告。提交成功。看起来一切干净。

这就是文档反复说的"零命中与干净结果不可区分"。仓库在多个地方记录了同一个 bug 的变体：`/security-audit` 的 Godot 专用 grep 在 Unity/Unreal 上返回零命中，被读成通过；本地化检查找不到硬编码字符串；内容审计数不到已实现内容；阶段检测数不到源文件，把完成的项目当成全新项目。

修法不是给每条 grep 加一个引擎分支，而是把代码根解析集中到一个函数 `resolve_code_root()`，让 Hook 和技能共用同一套语义。这样 Hook 和技能不会对"代码在哪"产生分歧。

引擎知识快照

`docs/engine-reference/` 是另一层适配。它解决的问题不是路径，是模型的知识截止日期。

目录结构固定：

```
<engine>/
├── VERSION.md                # 钉住的版本、验证日期、知识缺口窗口
├── breaking-changes.md       # 版本间的破坏性变更，按风险分级
├── deprecated-apis.md        # "不要用 X，用 Y" 对照表
├── current-best-practices.md
└── modules/                  # 每个子系统一份速查，控制在 150 行内
```

Godot 的 `VERSION.md` 写着：引擎版本 4.7，项目钉住日期 2026-10-17，文档最后验证 2026-10-17，模型知识截止 2025 年 5 月。知识缺口警告说：训练数据大概覆盖到 4.3，4.5、4.6、4.7 在截止之后发布，引入了

模型不知道的变化。

这份文件还记录了一个反向缺口，值得注意：

> 上面的警告是单向的，只说模型知道得比这个钉住版本少。反向缺口是真实存在的，而且 `/setup-engine` 的第 3 步会故意制造它（"钉新版本，之后再升级"）：参考资料可能超前于本机安装的编辑器，代理正确引用它，却生成本地编译不过的 API。信任带版本号的结论前，先看 `Installed at pin time`。`NOT DETERMINED` 意思是缺口未知，不是没有缺口。

这个区分很细：参考资料新于模型，是模型该查文档；编辑器旧于参考资料，是本地环境该升级。两种缺口的修法完全不同。

引擎专家

`project.yaml` 的 `specialists` 块有四个槽位：`primary`、`code`、`shader`、`ui`，外加 `additional`。

`primary` 从引擎名来：Godot 对应 `godot-specialist`，Unity 对应 `unity-specialist`，Unreal 对应 `unreal-specialist`。

`code` 是语言专家。这个槽位存在的理由是：`engine.name: Godot` 无法区分 GDScript 和 C#，而两者的写法差别很大。`/setup-engine` 会写入 `godot-csharp-specialist` 或 `godot-gdscript-specialist`。

`shader` 和 `ui` 按故事内容触发。`/dev-story` 在选择专家时，只在故事涉及着色器或材质时才带上 shader 槽位，只在 UI 故事里带上 ui 槽位。

所有槽位都是 `unset` 且没有 `primary` 时，技能说"未配置引擎专家"，然后继续，不阻塞。这是一个合理的降级：没有专家不等于不能写代码，只是少一层校验。

解析检查的引擎差异

`/dev-story` 对 Godot 给了完整命令，对其他引擎指向 `references/engine-checks.md`。Godot 的检查分两步：

```
godot --headless --path . --import
godot --headless --path . -s res://.claude/scripts/godot-parse-
check.gd -- res://<each changed .gd>
```

第一条导入资源，第二条逐个检查脚本能否加载。退出码 1 表示有脚本没加载，输出里的 `PARSE FAIL:` 点名是哪个。

技能明确禁止两件事：用 `--check-only` 代替，以及只跑 `--import` 就声称做过解析检查。两条都指向同一个失败模式：一个看起来跑了、实际没验证的检查。

没有引擎可执行文件时，结果写 `parse NOT VERIFIED — engine binary not available`。不许读代码推断"应该能解析"。

测试根也跟着引擎走

同一个坑出现在测试上：

引擎	测试根	放错地方的结果
Godot	`tests/unit/`、`tests/integration/`	不运行
Unity	`Assets/Tests/EditMode/`、`PlayMode/`	测试运行器显示 "0 tests, Passed"
Unreal	`Source/<Module>/Private/Tests/`	"No automation tests matched"

Unity 那一行尤其危险。`0 tests, Passed` 是一个绿色的成功信息，容易被当成通过。实际是测试文件没被编译，一条都没跑。

常见误解

误解一：代码根永远是 `src/`。它只是 Godot 那一行。

误解二：参考资料越新越好。参考资料新于本机编辑器时，会生成编译不过的代码。

误解三：引擎专家只影响提示词措辞。它决定审查哪些 API、用哪套惯例、检查哪些文件扩展名。

练习

1. 在副本里同时创建 `src/` 和 `Assets/`，运行 `project-state.sh`，观察 `code_root` 的来源和值。
2. 解释为什么 Unity 项目里把测试放到 `tests/` 会得到"0 tests, Passed"，这个结果为什么比报错更危险。
3. 假设模型建议了一个 Godot 4.7 的 API，而本机装的是 4.4，按仓库的做法应该怎么处理?

答案提示

1. 两个候选根同时存在属于有歧义，解析应返回未解析，`code_root` 一行会显示空值或 `unset` 来源，而不是任选一个。
2. Unity 只编译 `Assets/` 下的测试程序集，别处的文件不参与编译。运行器没有测试可跑，就报告零测试并通过。报错至少会让人注意，绿色通过会让人以为验证过了。
3. 先查 `docs/engine-reference/godot/VERSION.md` 的 `Installed at pin time` 和 `Knowledge Gap Warning`，标记该 API 为待验证，对照官方文档确认它在 4.4 是否存在，不确定就标 NOT SOURCEABLE 而不是照写。

14

上下文与恢复：把文件当作记忆

场景

一场设计讨论进行了两小时，你刚批准了战斗系统的第三节，模型正准备写文件，会话突然因为上下文超限中断。重开一个会话，它不记得刚才批准了什么，也不记得为什么否掉了另外两个方案。

如果所有决策只存在于对话里，这场讨论就白费了。CCGS 的应对是把"记忆"从对话搬到磁盘。

检查点文件

`production/session-state/active.md` 是会话检查点。模板里写了它的结构：

```
<!-- STATUS -->
Epic:
Feature:
Task:
<!-- /STATUS -->

<!-- CHECKPOINT -->
```

```
**Updated:** [YYYY-MM-DD]
**Branch:** `[branch]`
**Current task:** ...
**Next step:** ...
**Blocked on:** ...
**Files in progress:** ...
**Run result:** ...
**Open questions:** ...
<!-- /CHECKPOINT -->
```

文件里有两块机器读的区域，和一大块人读的叙述区。

`STATUS` 块给状态栏用，只在 Production 之后才有内容，显示 `史诗 > 功能 > 任务` 的面包屑。

`CHECKPOINT` 块每次会话启动时被 `session-start.sh` 原样展示，所以它有硬性长度限制：大约 25 行。它每次被覆盖，绝不追加。

`/CHECKPOINT` 之后的叙述区给人看，也给想要细节的代理看。没有 Hook 会注入它，所以它可以变长，但超过约 200 行时应该用 `rotate-session-state.sh` 移到 `production/session-logs/` 。

这个拆分解决了一个具体问题。如果一份只追加的文件同时承担两种职责，它会无限增长，而每个消费者读不同的切片：顶部、底部、或者一个没人写的块。谁都没错，因为没有契约规定可恢复状态在哪。有了命名区域，所有消费者读同一部分，成本固定。

写入时机

`context-management.md` 要求在几个里程碑后更新检查点：

- 一个设计小节批准并写入文件。
- 一个架构决策做出。
- 一个实施里程碑达成。
- 拿到测试结果。

`/dev-story` 和 `/story-done` 在每个结局都会写检查点，包括 BLOCKED 和 INCOMPLETE。这是恢复能力的关键：失败也要留下可恢复的状态。

检查点里的 `Next step` 有明确标准：要具体到能让一个冷启动的读者直接继续。`Run result` 那一行由 `/dev-story` 原样写入，供 `/story-done` 读取。

增量写文件

`context-management.md` 的另一条建议针对多章节文档：

1. 先问"我可以创建 [路径] 和章节骨架吗"，创建带空标题的文件。
2. 在对话里讨论并起草一个小节。
3. 一旦批准立即写入文件。
4. 写完后更新会话状态。
5. 之前关于该小节的讨论可以安全压缩。

效果是把上下文占用从"整份文档的讨论历史"（三到五万 token）降到"当前小节"（三到五千 token）。

这条建议的价值不限于这套框架。任何长文档协作都可以用：文件是持久层，对话是工作区，批准后立刻落盘。

压缩策略

文档给出的阈值是：上下文用到 60% 到 70% 时主动压缩，不要等撞上限。`/clear` 用于不相关任务切换，或者连续两次纠正失败之后。

压缩时要求保留的内容包括：会话状态文件路径、本次修改的文件清单、架构决策及理由、当前任务和所处步骤、哪些章节已写入文件、哪些还在讨论中。

压缩之后的第一件事是读 `active.md` 和正在处理的文件。文件里有决策，对话历史是次要的。

这套做法的前提是模型愿意在压缩时保留这些东西。它是一个提示，不是保证。真正可靠的是文件本身已经写好了。

子代理与上下文预算

`context-management.md` 给了三档粗略预算：轻量阅读约 3k token 启动，实现功能约 8k，多系统重构约 15k。

并给了子代理使用建议：跨多个文件调查、探索不熟悉的代码、研究会消耗 5k 以上 token 的读取时，用子代理；已经知道要读哪一两个文件时，直接读。

子代理不继承对话历史，所以提示词必须自带完整上下文。这一点和团队流水线的返回契约配合：父会话只给蒸馏后的简报，子代理只返回路径和摘要。

一个被反复强调的原则

`CLAUDE.md` 把两件事标为"承重"，其中一件是：

> `.claude/scripts/` 里的辅助脚本只输出观察，不输出判决。

理由写得很清楚：一个会打分或下判断的脚本，迟早会和它看不到的模式、层级、按系统覆盖相矛盾。脚本不知道当前是 lean 还是 full，也不知道某个系统被单独提到了 full，它给出的"通过"会误导调用方。

这条原则和恢复机制是同一个思路的两面：确定性的部分交给脚本，判断交给能看到全局的技能和用户。脚本报告"文件存在"、"哈希变了"、"注册表有三条重复"，技能决定这意味着什么。

恢复演练

真正可靠的恢复流程需要演练。仓库给的路径是：

1. 会话崩了，重开。
2. `session-start.sh` 检测到 `active.md`，预览检查点。
3. 读完整状态文件和正在处理的文件。
4. 从下一个未完成的小节继续。

要验证这条链，可以手动做一次：写一个检查点，清空上下文，重开会话，看预览是否出现、内容是否足够让你继续。如果没有演练过，恢复能力只是假设。

常见误解

误解一：模型会记住上一场会话。不会。所有跨会话记忆都靠文件。

误解二：检查点越长越好。机器读的部分有 25 行上限，超了会被截断或挤占上下文。细节应放叙述区或日志。

误解三：压缩会自动保留重要内容。压缩是摘要，可能丢细节。重要决策要在压缩前写进文件。

练习

1. 按模板创建一份 `active.md`，写一个具体的 `Next step`，要求冷读者能直接执行。
2. 模拟一次恢复：只读检查点，写出你接下来要做的三个动作。
3. 解释为什么辅助脚本不应输出"通过"或"失败"，而应输出事实。

答案提示

1. 例如"实现 `src/combat/parry.gd` 的 `can_parry()`，测试在 `tests/unit/combat/parry_test.gd`，先跑 `godot --headless --path . --import`"。要包含文件、验证命令、当前阻塞。
2. 典型三步：读状态文件确认范围，读进行中的文件确认已写到哪，执行 `Next step` 里的命令。
3. 脚本看不到配置层级、按系统覆盖、当前模式。它输出"通过"时，实际是在一个它不掌握的规则集下做判断，容易和技能或用户的理解冲突。事实不会过时，判决会。

15

安全边界：能聊天不等于能管理

场景

你克隆了一个模板，输入 `claude`，它开始执行仓库里的 Bash 脚本。这些脚本会读你的 git 状态、扫描暂存文件、写日志、可能发桌面通知。它们运行在你的用户权限下，不需要额外授权。

从安全角度看，克隆这个仓库等于安装了一个本地开发工具。它没有服务端，没有远程执行，但确实有代码在你机器上跑。理解它的信任模型，比记住"有 15 个 Hook"更重要。

信任模型

`SECURITY.md` 把范围写得很清楚。这是一个本地开发工具，安装 shell Hook，协调直接在你机器上运行的 AI 代理。安全问题的核心是"贡献的代码在用户环境里执行，而用户不知情"。

高严重度问题包括：

- Hook 执行恶意或未披露的 shell 命令。
- 技能或角色泄露环境变量、API key、密钥。

- 通过技能或角色定义做提示注入，让 Claude 绕过安全措施或执行未授权的破坏性操作。
- 贡献静默改变行为，用户无法审计。

中严重度包括：未披露的外发网络请求、角色定义提升权限或绕过确认、跨平台行为不一致的 Hook 模式、技能在文档范围外写文件。

不在范围内：Claude Code 本身的行为、用户安装问题、没有现实攻击路径的理论漏洞、需要物理接触的攻击。

这份清单本身就是一份很好的威胁建模模板。它区分了"代码执行"、"数据外泄"、"权限提升"、"行为隐蔽"四类风险。

权限层能挡什么

`settings.json` 的 `permissions.deny` 列了很长的模式。按类别看：

- 破坏性删除：`rm -rf` 的二十多种参数组合、`rm --recursive --force`、`git clean -f` 的多种变体。
- 强制推送：`git push --force`、`git push -f`、`git push +`，包括前置 `git -C` 的形式。
- 权限提升：`sudo`、`chmod 777`。
- 敏感文件：`Read(**/.env*)`、`Edit(**/.env*)`、`PowerShell(Get-Content *.env*)`。
- 二进制资产：`Edit(**/*.uasset)`、`Edit(**/*.umap)`。

这层由宿主执行，比提示词硬。但它是命令字符串匹配，有几个天然边界。

第一，只覆盖列出的形式。`find . -delete`、`truncate -s 0`、覆盖写`> file`都不在列表里。

第二，跨工具边界。`Read(**/.env*)`拦的是 Read 工具，`Bash(cat .env)`是另一条路径。除非 deny 列表里也有对应的 Bash 模式，否则读得到。

第三，模式依赖于解析的准确性。`validate-push.sh`花了近三百行处理`&&`链接、引号、`git -C`、`env`前缀、PowerShell here-string，正说明"判断这是不是一次推送"本身就不简单。

Hook 层能挡什么

第 10 章已经拆过：15 个注册命令里只有 3 处会阻断。

`validate-read.sh`拦超大整篇读取，属于上下文保护，不是安全边界。

`validate-assets.sh`在写入后报告无效 JSON，无法撤销。

`validate-commit.sh`只在数据 JSON 校验失败时阻断提交。

其余 Hook 都是提醒或记录。`validate-write.sh`检查子代理是否越界写文件，发现问题时打印警告，退出码 0。

这意味着：如果一个子代理被提示注入，试图写 `production/` 之外的文件，Hook 会提醒，但不会阻止。真正的阻止要靠权限层或用户在看到警告后介入。

自动化模式与不可省的问题

`automation-modes.md` 定义了三种模式：collaborative 每次问、guided 只问重大决策、autonomous 不问但记录。

autonomous 看起来最危险，但它保留了 `automation_always_ask` 列表。默认三类：范围变更、文件删除、schema 变更。

这个列表的设计意图是：把不可逆或影响面大的决策固定在人的手里，其余交给模型。用户可以在 `project.yaml` 里扩展列表，比如加上外部 API 调用、版本升级、架构决策。

文档还强调：这个列表是"你可以配置的类别"，不是"永远会问的类别"。只有列表里实际存在的类别才会打断 autonomous 运行。把没配置的类别当成生效，会误报违规。

不往上游提交

`coordination-rules.md` 第 7 条：

> 永不向引擎、插件或供应商仓库提交、开启或评论 issue 和 pull request，即使被要求。只写草稿（`/bug-report` 只起草），由用户提

> 交。

这条规则防的是身份冒用。代理以用户身份对外发声，一旦内容有误或包含敏感信息，后果由用户承担。起草加人工提交，把对外动作留给人。

从安全角度看，这也是提示注入的一道闸：即使某个被读取的文档指示"去给上游提 issue"，规则明确禁止执行。

数据外泄面

技能和角色可以调用 `WebSearch` 的角色有 9 个（带 WebSearch 的 6 个加带 Bash 和 WebSearch 的 3 个）。理论上，任何能读文件又能发网络请求的代理都可以外传内容。

仓库的对策是审计和披露，不是技术隔离：

- `SECURITY.md` 要求贡献者不得有未披露的网络调用。
- 技能头部声明允许的工具，`allowed-tools` 精确到子命令。
- `.claude/rules/` 限制可写路径。
- 提交前 Hook 扫描数据文件和硬编码值，但不扫描外发请求。

如果你在企业环境使用这套模板，需要自己补一层网络出口控制。仓库没有提供。

更新通道

模板通过 `UPGRADING.md` 从上游拉取更新。这意味着上游的脚本变更会进入你的项目。一个被篡改的 Hook 会在你下次运行 Claude Code 时执行。

这是所有模板类项目的共同风险。可行的缓解是：升级前看 diff，尤其是 `.claude/hooks/` 和 `.claude/scripts/`；固定到你审查过的提交；不要盲目跟随 `main`。

安全审计技能

仓库自带 `/security-audit`，它检查的是游戏项目的安全，不是模板本身。范围包括：存档篡改、凭据暴露、服务端权威绕过、硬编码密钥、第三方依赖清单。

高严重度定义是：绕过进度的存档篡改、凭据暴露、服务端权威绕过。这提醒读者：这套模板主要用于单机游戏，多人游戏的服务端权威、反作弊、网络验证不在它的核心覆盖里，需要额外的架构和审计。

常见误解

误解一：deny 列表等于沙箱。它只匹配列出的命令形式，且不覆盖所有工具路径。

误解二：autonomous 模式无人监督。它保留了三类强制提问，且所有决策进日志。

误解三：模板代码天然可信。它会在你的机器上执行，信任应基于审查和固定版本，而不是"开源所以安全"。

练习

1. 从 `settings.json` 的 deny 列表里找出三种没有被覆盖的破坏性命令，并说明如何补。
2. 设计一个提示注入场景：一个被读取的设计文档要求代理把 `.env` 内容写进公开的变更日志。仓库里哪几条规则会阻止或暴露它?
3. 解释为什么 `automation_always_ask` 默认包含"文件删除"而不包含"创建文件"。

答案提示

1. 例如 `find . -delete`、`truncate -s 0 file`、`> file`。补法是增加对应的 Bash 模式，或把破坏性操作收敛到脚本里统一审计。命令字符串匹配永远有遗漏，重要的是知道边界在哪。
2. 权限层拦 Read 工具的 `.env`，但不拦 `Bash(cat .env)`；`validate-write.sh` 会提醒写入受保护文件；`/story-done` 的范围检查会把不在 `## Files` 里的改动标为 OUT OF SCOPE；评审环节可能发现。没有任何单层能保证拦住，这就是分层的原因。
3. 删除不可逆且影响历史记录，创建可逆且容易审查。风险不对称，所以默认只把删除固定在人的手里。

16

验证与演进：这门框架怎么证明自己没坏

场景

你改了 `dev-story` 的一个阶段，加了两条规则。怎么知道没把技能改坏?

普通软件有编译器、单元测试、集成测试。技能是 Markdown，没有编译器。CCGS 的答案是一套针对技能本身的静态检查和行为规格。

studio-tests

`studio-tests/` 是"测试技能和角色"的目录，不是测试游戏的。
README 说得很明确：随模板发布，也理应随模板发布，因为模板本来就是给你改的。

内容分四块:

- `catalog.yaml`： 74 个技能和 49 个角色的主登记表，记录覆盖率。
- `quality-rubric.md`： 按类别的通过标准。
- `skills/`： 每个技能一份行为规格，按 gate、review、authoring、readiness、pipeline、analysis、team、sprint、utility 分类。

- `agents/`：每个角色一份规格，按总监、主管、专家、引擎、运营、QA 分类。

驱动它的是两个技能：`/skill-test` 和 `/skill-improve`。

四种测试模式

`/skill-test` 有四个模式：

`static` 做七项结构检查，读技能文件，不运行技能。可以测单个，也可以 `all`。

`spec` 拿行为规格逐条评估技能或角色。比如 `gate-check` 的规格会检查它是否读取评审模式、是否在 solo 模式跳过全部门禁、是否禁止自动推进。

`category` 按类别标准评估。gate 类技能的关键指标是"读取评审模式、full/lean/solo 三种总监小组、不自动推进"。

`audit` 输出覆盖率全景：哪些技能有规格、上次测试时间、结果。

README 还给了一张"删掉 studio-tests 会怎样"的表：static 仍能跑，audit 退化为报告没有目录，spec 和 category 直接失效，`/skill-improve` 失去类别通过项。这张表是很好的依赖说明范例：说清每个模式的具体代价，而不是笼统说"会影响测试"。

静态检查的实测

`skill-lint.sh` 是 static 模式七项检查的脚本实现。在快照上对 `dev-story` 运行：

```
skills     1          .claude/skills
skill      dev-story  files=3    lines=389
check1     dev-story  frontmatter=ok        missing=-  name=match
check2     dev-story  phases=7   h2=11      h2_total=11
check3     dev-story  verdicts=FAIL:1,BLOCKED:6,COMPLETE:1
assessment_words=readiness
check4     dev-story  may_i_write=1         before_writing=0
approval=0 ask_write_lines=5     write_tools=Write,Edit
check5     dev-story  last_section=Phase 7: Summary, files,
checkpoint skills_named=story-done,dev-story,code-
review,architecture-decision     next_phrases=6/5
check6     dev-story  context_fork=no       phases=7   h2=11
check7     dev-story  hint=[story-path]     args_section=-
checked    1 of 1
```

七项分别看：前置元数据完整、阶段数、判决词、写入前是否请求批准、最后一节是否指向下一步、是否设置独立上下文、参数提示。

对全部技能运行，最后一行是 `checked 74 of 74`。这个数字很重要：它证明脚本读到了全部 74 个，而不是读了 60 个然后报告完成。脚本头部专门说明，读不到的技能会打印 `NOT_READ`，且不计入已读，避免"没检查看起来像通过"。

注意脚本输出的定位：它只给事实，不给判决。`check3` 列出判决词出现次数，至于 `BLOCKED` 出现六次是否过多，由 `/skill-test` 判断。这符合 `CLAUDE.md` 里"辅助脚本只输出观察"的承重约定。

五条义务

`.claude/rules/skill-authoring.md` 是写技能和门禁时的核心规则，只有五条，但每条都指向同一类失败。

第一条：每个门禁判决都要有"无法评估"值。三态判决会把所有未知压成一个断言。排序要讲究：它必须高于通过，低于失败。

第二条：缺失值不能默认成宽松值。尤其在框架里没人写那个键的时候。先问谁设置它；一个有读无写的键，默认分支就是每个项目的实际行为。

第三条：跳过的步骤要自己说出来。在输出里说，不只写在源码注释里。

第四条：不要断言无法溯源的事实，并且把缺口接到判决上。只标记不接线，缺口就只是注释。

第五条：门禁要派生覆盖集，不要枚举。白名单会漏掉后来新增的技能。如果类别无法机械判定，就在门禁里说明原因并保留清单，同时记录扩展义务。

还有两条门禁细则：断言一个不可能偶然为真的东西，比如不要检查单词 "unity"，因为文件路径里可能包含它，而 "community" 也包含；以及"没亲眼见它失败过的门禁不算门禁"，要故意破坏一次，确认它失败，再恢复。

最后一条最容易被跳过。写了一个检查，跑一次显示通过，就以为它能工作。实际上它可能永远通过。仓库要求做一次变异测试：把被守护的东西改坏，看检查是否报警。

版本与升级

`project.yaml` 里有 `framework.version: 1.2.0` 和 `last_upgraded: 2026-10-19`。`UPGRADING.md` 描述从上游拉取更新的流程，`migrate-v1-config.sh` 处理旧配置迁移。

这套机制的风险在第 15 章讨论过：模板更新会带入可执行脚本。从工程角度，升级还需要三件事：

1. 升级前看 `.claude/hooks/` 和 `.claude/scripts/` 的 diff。
2. 升级后跑 `/skill-test static all`，确认 74 个技能都能读。
3. 跑一次真实的 `/dev-story` 或团队流水线，确认配置解析和行为规格没有漂移。

只有第一步是自动的。后两步需要真实项目。

这套验证覆盖不到什么

`skill-test` 的所有模式都只读文件，不运行技能。README 明确写了：每个模式都读文件，没有一个运行技能。所以它证明的是结构合规和规格存在，不是行为正确。

真正的行为验证需要真实会话。仓库在 `studio-tests/catalog.yaml` 里记录"上次规格测试"和"上次类别测试"的时间与结果，但这些记录也依赖人手动触发。

还有一类缺口：文档提到的根目录 `tests/` 在快照里不存在。`release-review` 文档说框架自己的测试套件覆盖了配置解析链，但这次克隆的仓库里没有那个目录。这不影响使用，但说明"有测试"和"测试在当前分发里"是两件事。读的时候要区分。

常见误解

误解一：`/skill-test static` 通过就说明技能能正常工作。它只检查结构，不执行技能。

误解二：规格测试是自动化的。它需要模型读规格、逐条判断，结果依赖模型的理解。

误解三：`studio-tests/` 是可选装饰。删掉它，spec 和 category 两个模式直接失效。

练习

1. 对任意一个技能运行 `skill-lint.sh`，解释七项检查各自的用途。
2. 为一个你写的门禁设计一条"无法评估"路径，并说明它在判决排序中的位置。
3. 选择一个现有检查，做一次变异测试：破坏它守护的对象，确认它失败，再恢复。

答案提示

1. 前置元数据完整性、阶段结构、判决词覆盖、写入前批准、结尾指向下一步、上下文隔离、参数提示。它们共同保证技能可被发现、可被执行、可被交接。
2. 例如"文件存在但无法读取"应返回 NOT ASSESSED，而不是通过或失败。它排在通过之上、失败之下，理由是无法评估没有建立良好证据，但已知失败更可操作。
3. 例如把一份 JSON 数据文件故意写坏，运行 `validate-commit.sh` 的对应路径，确认得到 `exit 2`；恢复文件再跑一次确认通过。

17

最小实验：用一百多行 Python 复刻四条约束

场景

读完十多万字的规则和几十个 Bash 脚本，很多机制仍然停留在"看懂了"的层面。要确认真的理解，最好的办法是用另一种语言把它重写一遍。

这一章给出一个 `examples/mini_studio.py`，约两百行，标准库，无依赖。它不复制上游脚本，只复刻四个决定行为走向的约束：配置解析、故事状态、验收判决、证据时效。配套 17 个单元测试，全部通过。

配置解析

模型里的 `resolve()` 接收三张表：项目配置、本地配置、遗留配置。

```
def resolve(project, local=None, legacy=None):
    ...
    for key in sorted(keys):
        for source, table, allowed in (("project.local.yaml", local,
LOCAL_KEYS),
                                       ("project.yaml", project,
None),
```

```
                                        ("legacy", legacy, None)):
            if allowed is not None and key not in allowed:
                continue
            if key in table:
                if _valid(key, table[key]):
                    out[key] = (table[key], source)
                    break
                notes.append(f"{key}: '{table[key]}' is not a valid
enum value; ignored")
        else:
            # rigor expansion, then documented default
```

它保留了三个关键行为：

1. 本地配置只对白名单键生效。测试 `test_local_ignored_for_protected_key` 把 `qa.level` 同时写进两张表，断言结果来自项目配置。
2. 显式值优先于 rigor 展开。测试 `test_explicit_key_beats_rigor_expansion` 在 `rigor: full` 下显式写 `workflow: minimal`，断言 workflow 保持 minimal。
3. 非法枚举回落而不是胜出。测试 `test_invalid_enum_falls_through` 用 `rigor: strict`，断言 workflow 变成 minimal，并且备注里点名了非法值。

第三个行为最容易被写错。直觉写法是"读到就用"，但那样一个拼写错误会传播到所有技能，而错误配置往往比缺省配置更隐蔽。

代码根解析

```
def resolve_code_root(engine, dirs):
    by_engine = {"godot": "src", "unity": "Assets", "unreal":
"Source"}
    if engine in by_engine:
        return by_engine[engine], "engine.name"
    candidates = [d for d in ("src", "Assets", "Source") if d in
dirs]
    if len(candidates) == 1:
        return candidates[0], "detected"
    return None, "unresolved"
```

三个测试覆盖三种情况：引擎优先、单一目录推断、两个候选目录时不猜。

第三个断言返回 `(None, "unresolved")`，而不是默认到 `src`。这是第 13 章那条规则的代码化：缺失值不能默认成宽松值。

故事状态与"跳过不是同意"

```
def start(self):
    if self.status not in ("Ready", "Blocked"):
        return f"refused: {self.status} cannot start"
    if self.answer not in ("yes", "approved"):
```

```
        return "waiting: no explicit approval"
    self.status = "In Progress"
    return "started"
```

`answer` 模拟用户对"我可以开始吗"的回复。只有明确同意才推进，`None` 、空字符串、 `"skipped"` 都停在原地。

对应的测试是 `test_skipped_question_is_not_approval` ： 调用 `start()` ， 断言返回包含 `waiting` ， 且状态仍然是 `Ready` 。

关闭也有守卫： `close()` 只在判决为 PASS 且状态是 In Progress 时生效。 `test_no_close_on_unknown` 断言 UNKNOWN 判决被拒绝，状态保持 In Progress。

判决排序

```
PASS, NOTES, UNKNOWN, FAIL = "PASS", "PASS_WITH_NOTES", "UNKNOWN",
"FAIL"
RANK = {FAIL: 3, UNKNOWN: 2, NOTES: 1, PASS: 0}


def verdict(criteria):
    states = {state for _, state in criteria}
    if "FAILS" in states:
        return FAIL
    if "UNTESTED" in states:
        return UNKNOWN
```

```
    if "DEFERRED" in states:
        return NOTES
    return PASS
```

四个测试覆盖：失败压过未知、未知压过通过、延期算"带备注的通过"、全部覆盖算通过。

`RANK` 在演示里没有直接用，它表达的是排序关系：FAIL 最高，UNKNOWN 次之，NOTES，PASS。这正是 `skill-authoring.md` 第一条义务要求的排序：无法评估要高于通过，低于失败。

证据时效

```
def digest(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()[:12]


def evidence_state(stamp, paths):
    if not stamp:
        return ["RECEIPT_NONE"] + [f"NEW: {p}" for p in paths]
    ...
```

`receipt()` 给文件打内容哈希， `evidence_state()` 比对，输出五种状态：UNCHANGED、CHANGED、NEW、GONE、RECEIPT_NONE。

测试里创建临时文件、打回执、修改、删除，依次断言 UNCHANGED、CHANGED、GONE。另一个测试断言没有回执时第一行是 RECEIPT_NONE，而不是空列表。

最后一条很重要：没有回执时不能返回空结果，因为空结果会被读成"没有变化"。必须显式说"无法判断"。

编号不回收

```
def next_id(kind, seen):
    nums = [int(x.rsplit("-", 1)[-1]) for x in seen if x.rsplit("-",
1)[-1].isdigit()]
    return f"{kind}-{max(nums, default=0) + 1:03d}"
```

测试给 `["story-001", "story-007"]`，断言结果是 `story-008`，不是 `story-002`。

这条规则的成本是编号会稀疏，收益是历史引用永远指向同一对象。

实际运行

```
$ cd examples
$ python3 -m unittest -v test_mini_studio.py
...
Ran 17 tests in 0.001s
OK
```

```
$ python3 mini_studio.py demo
config: {'docs.density': ('balanced', 'rigor:standard'), ...,
         'modes.review_mode': ('solo', 'project.local.yaml'), ...}
invalid enum: ('minimal', 'rigor:minimal')
code root: ('Assets', 'engine.name')
code root ambiguous: (None, 'unresolved')
start without answer: waiting: no explicit approval
start with answer: started
close on UNKNOWN: refused: verdict=UNKNOWN status=In Progress
close on PASS: closed
verdict mixed: PASS_WITH_NOTES
verdict untested: UNKNOWN
evidence fresh: ['UNCHANGED: /tmp/.../combat.gd']
evidence stale: ['CHANGED: /tmp/.../combat.gd']
ids: story-008
```

输出本身就是一份行为清单。配置里 `review_mode` 来自本地文件，其余来自 standard 展开；非法 rigor 被忽略并回落到 minimal；两个候选目录被视为未解析；没有明确回答时故事不启动；UNKNOWN 不允许关闭；证据修改后被识别为过期。

与真实实现的差距

这个模型刻意简化了几件事：

- 上游用 Bash 和 awk 实现，为了在没有 Python 的环境里也能跑；模型直接用 Python。
- 上游的配置解析还处理旧版镜像、Tab 缩进、BOM、CRLF、重复块合并；模型只保留优先级和枚举回落。
- 上游的故事状态由技能流程维护，没有代码强制；模型把它写成了显式状态机，反而更严格。
- 上游的证据检查用 git hash-object 或 sha1sum；模型用 sha256，语义相同。

这些差距本身是练习材料。把任何一项补上，都会暴露新的边界情况。

练习

1. 给 `resolve()` 加旧版镜像支持，让 `review_mode` 可以从 `legacy` 表读到，并写一个测试断言优先级低于项目配置。
2. 给 `Story` 加 `Blocked` 状态和一条解除阻塞的路径，要求解除时必须有明确的用户回答。
3. 把 `evidence_state()` 扩展为支持目录参数，目录下的文件逐个比对，写测试覆盖新增文件和删除文件。

答案提示

1. 在循环里插入 `("legacy", legacy, None)` 一级，注意放在项目配置之后、展开之前；测试里同时写项目值和遗留值，断言来源是 `project.yaml` 。

2. `unblock(reason)` 只在 `status == "Blocked"` 且 `answer` 是明确同意时转为 `Ready`，其余返回 `waiting`。
3. 用 `Path.rglob("*")` 展开目录，把相对路径作为键；新增文件报 NEW，回执里有但磁盘没有的报 GONE。测试可以用 `TemporaryDirectory` 创建两层结构。

A

附录：源码索引、术语与核验路线

快照

字段	值
仓库	Donchitos/Claude-Code-Game-Studios
提交	05ad7c8a5b8aa2aa86bbd3439ae7cf7295cac418
日期	2026-10-19
框架版本	1.2.0
许可	MIT
本书采集日期	2026-10-31

每章源码入口

章	主要文件
前言	`README.md`、`LICENSE`、`project.yaml`
01 需求	`CLAUDE.md`、`docs/COLLABORATIVE-DESIGN-PRINCIPLE.md`、`.claude/docs/coding-standards.md`、`.claude/rules/*.md`

章	主要文件
02 架构	`.claude/settings.json`、`.claude/docs/directory-structure.md`、`.claude/docs/context-management.md`
03 入门	`.claude/skills/start/SKILL.md`、`.claude/scripts/project-state.sh`
04 配置	`.claude/docs/config-resolution.md`、`.claude/hooks/yaml-helper.sh`、`.claude/docs/settings-guidance.md`
05 角色	`.claude/agents/*.md`、`.claude/docs/agent-roster.md`、`.claude/docs/model-tiers.md`、`.claude/docs/coordination-rules.md`
06 技能协议	`.claude/skills/dev-story/SKILL.md`、`.claude/docs/automation-modes.md`、`.claude/rules/skill-authoring.md`
07 产物	`.claude/docs/templates/story.md`、`docs/architecture/tr-registry.yaml`、`design/registry/entities.yaml`、`.claude/docs/director-gates.md`
08 实施	`.claude/skills/dev-story/SKILL.md`、`.claude/docs/code-root-resolution.md`、`.claude/docs/run-and-observe.md`
09 验收	`.claude/skills/story-done/SKILL.md`、`.claude/scripts/review-receipts.sh`、`.claude/docs/error-recovery-protocol.md`

章	主要文件
10 Hook	`.claude/settings.json`、`.claude/hooks/*.sh`、`.claude/scripts/write-guard.sh`、`.claude/scripts/read-guard.sh`
11 注册表	`.claude/scripts/registry.sh`、三份注册表 YAML
12 协作	`.claude/skills/team-combat/SKILL.md`、`.claude/docs/coordination-rules.md`、`.claude/docs/error-recovery-protocol.md`
13 引擎	`.claude/docs/code-root-resolution.md`、`docs/engine-reference/**`、`.claude/docs/technical-preferences.md`
14 上下文	`.claude/docs/context-management.md`、`.claude/docs/templates/session-state.md`、`.claude/scripts/session-state.sh`
15 安全	`SECURITY.md`、`.claude/settings.json`、`.claude/skills/security-audit/SKILL.md`
16 验证	`studio-tests/README.md`、`studio-tests/catalog.yaml`、`.claude/scripts/skill-lint.sh`、`.claude/skills/skill-test/SKILL.md`
17 实验	`examples/mini_studio.py`、`examples/test_mini_studio.py`

实测数据

以下数字来自快照的实际统计和脚本运行，不是文档转述。

```
.claude/agents/*.md        49
  model inherit            30
  model sonnet             16
  model opus                3
.claude/skills/*/SKILL.md  74
.claude/rules/*.md         14
.claude/hooks/*.sh         18
  registered commands      15
.claude/docs/templates/*   71
```

`skill-lint.sh all` 的最后一行是 `checked 74 of 74`。

`project-state.sh` 在未配置的快照上报告：`engine unset`、`code_root src detected`、`source_files 0`、`gdd_files 0`、`adr 0`、`stage unset`、`phase_inferred concept`。

`resolve_config` 的默认输出：rigor minimal，workflow minimal，qa.level minimal，review_mode solo，team.size individual，automation collaborative。

`registry.sh validate`：三份注册表各 0 条目，五个检查项全为 0。

术语

Agent（角色）：`.claude/agents/` 下的 Markdown 文件，定义职责、工具、模型、回合上限。派生成子代理后运行。

Skill（技能）：`.claude/skills/<name>/SKILL.md`，斜杠命令的正文，包含前置元数据、注入命令和分阶段流程。

Hook：`.claude/hooks/` 下的脚本，由 `settings.json` 注册到事件上，读标准输入 JSON，用退出码影响行为。

Rule（规则）：`.claude/rules/` 下按路径生效的编码标准，编辑匹配路径时加载。

rigor：一个前置设置，展开成六个子设置。默认 minimal。

workflow：决定设计文档要写到多细，以及实施前必须存在哪些产物。

qa.level：决定需要什么测试证据，与 workflow 分开设置。

review_mode：full、lean、solo，决定总监评审门禁跑到什么程度。

team.size：individual、small、studio，决定团队流水线派生多少角色。

Story（故事）：最小执行单元，含验收标准、证据要求、文件范围、依赖。

ADR：架构决策记录，带状态和版本，故事引用其摘要。

TR-ID：需求编号，权威文本在 `docs/architecture/tr-registry.yaml` 。

Receipt（回执）：内容哈希戳，用于判断证据是否比代码旧。

NOT ASSESSED：无法评估，排在通过之上、失败之下。

核验路线

想自己验证书中的结论，按这个顺序：

1. 克隆固定提交，统计文件数：

```
git clone https://github.com/Donchitos/Claude-Code-Game-Studios.git
cd Claude-Code-Game-Studios
git checkout 05ad7c8a5b8aa2aa86bbd3439ae7cf7295cac418
find .claude/agents -name '*.md' | wc -l
find .claude/skills -name SKILL.md | wc -l
```

2. 看配置解析：

```
bash .claude/hooks/yaml-helper.sh resolve_config
bash .claude/hooks/yaml-helper.sh resolve_setting modes.workflow
```

3. 看项目状态和注册表：

```
bash .claude/scripts/project-state.sh
bash .claude/scripts/registry.sh validate
```

4. 看技能结构：

```
bash .claude/scripts/skill-lint.sh dev-story
bash .claude/scripts/skill-lint.sh all | tail -1
```

5. 看证据回执：

```
f=$(mktemp); echo a > "$f"
bash .claude/scripts/review-receipts.sh hash "$f" > "$f.stamp"
bash .claude/scripts/review-receipts.sh check "$f.stamp"
echo b >> "$f"
bash .claude/scripts/review-receipts.sh check "$f.stamp"
```

6. 跑本书的实验：

```
cd examples
python3 -m unittest -v test_mini_studio.py
python3 mini_studio.py demo
```

7. 官方语义对照：Claude Code Hooks 文档（`https://code.claude.com/docs/en/hooks`）用于确认退出码和事件行为。Hook 语义会随版本变化，移植前以你当前版本的文档为准。

未覆盖的问题

本书没有回答这些问题，它们需要真实项目和真实计费：

- 一次完整游戏开发的实际 token 成本。
- standard 与 full 档位在真实项目上的质量差异。
- 多代理并行时的冲突率和返工率。
- 模型对这套长提示词的实际遵循率。
- 在 Unity 或 Unreal 项目上的端到端表现。

作者自己的对照实验只覆盖了四个小游戏和一次盲评，样本很小，书中引用时保留了它的原始结论和局限。

上游许可与本书声明

上游模板采用 MIT 许可。本书是独立撰写的静态研究，不是上游官方文档，也不代表作者观点。书中的批评、改进建议和实验代码由本书作者负责。阅读器样式与构建脚本参考了本地电子书项目，仅用于排版与检索。